

EECS 836: Machine Learning

Zijun Yao

Assistant Professor, EECS Department
The University of Kansas

Agenda

- Self-Attention Networks (Transformer)
 - Why use self-attention for seq2seq processing
 - Transformer network details

Transformer

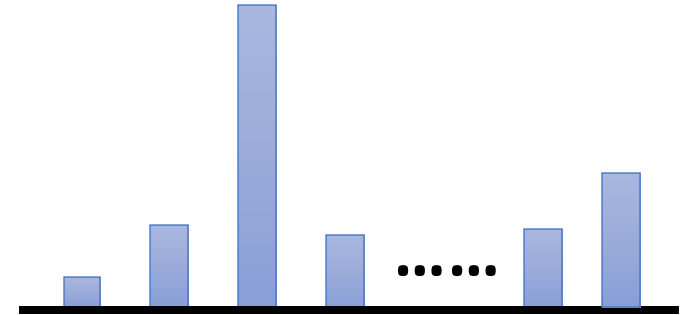


Transformer overview

This is a generative AI

Language
Model

Introduction



1.Tokenization

2. Input
Layer

3. Attention

4. Feed
Forward

5. Output
Layer

Word to Token

Understand Token

Understand Context

Aggregate

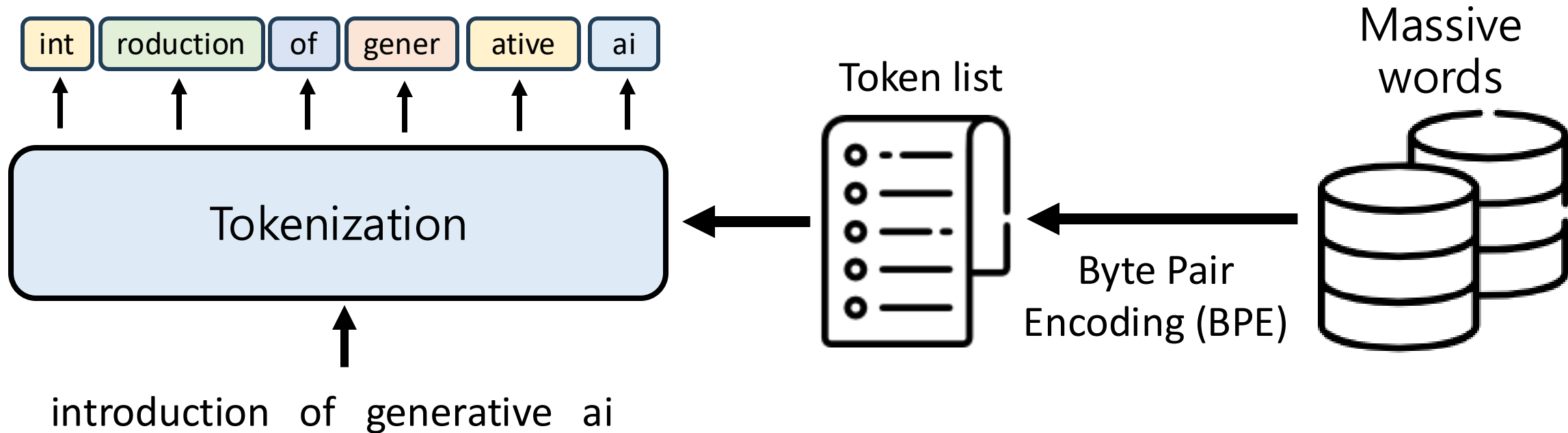
Get
Output

Transformer Block

Step 1: Convert text to tokens

- Language models process text in units called “tokens”

Tokenization has no trainable parameters



GPT-3.5 & GPT-4 GPT-3 (Legacy)

A language model is a probabilistic model of a natural language. In 1980, the first significant statistical language model was proposed, and during the decade IBM performed 'Shannon-style' experiments, in which potential sources for language modeling improvement were identified by observing and analyzing the performance of human subjects in predicting or correcting text.



Tokens

65

Characters

373

A language model is a probabilistic model of a natural language. In 1980, the first significant statistical language model was proposed, and during the decade IBM performed 'Shannon-style' experiments, in which potential sources for language modeling improvement were identified by observing and analyzing the performance of human subjects in predicting or correcting text.

Text Token IDs

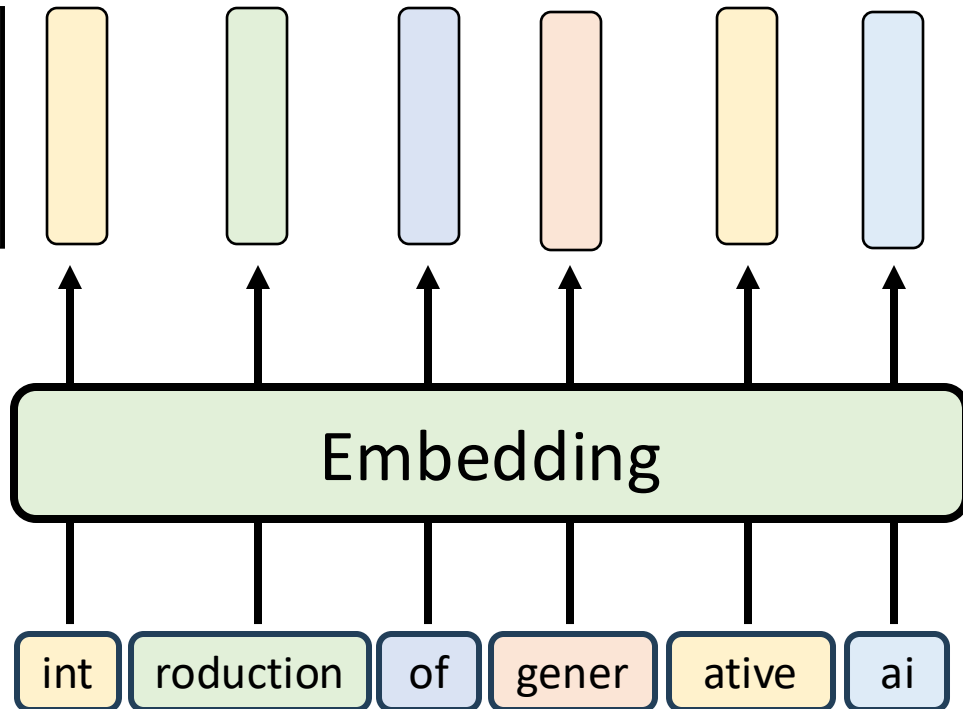
<https://platform.openai.com/tokenizer>

Step 2: Understand each token

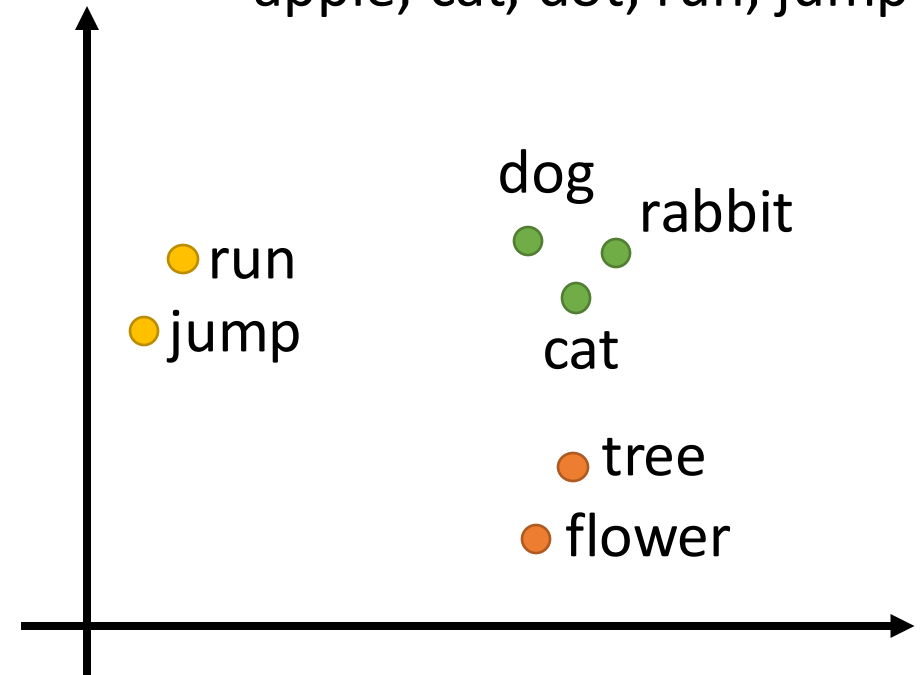
Tokens with similar meanings have similar embeddings.

- dog, cat, rabbit → similar vectors
- jump, run → similar vectors
- flower, tree → similar vectors

Vector

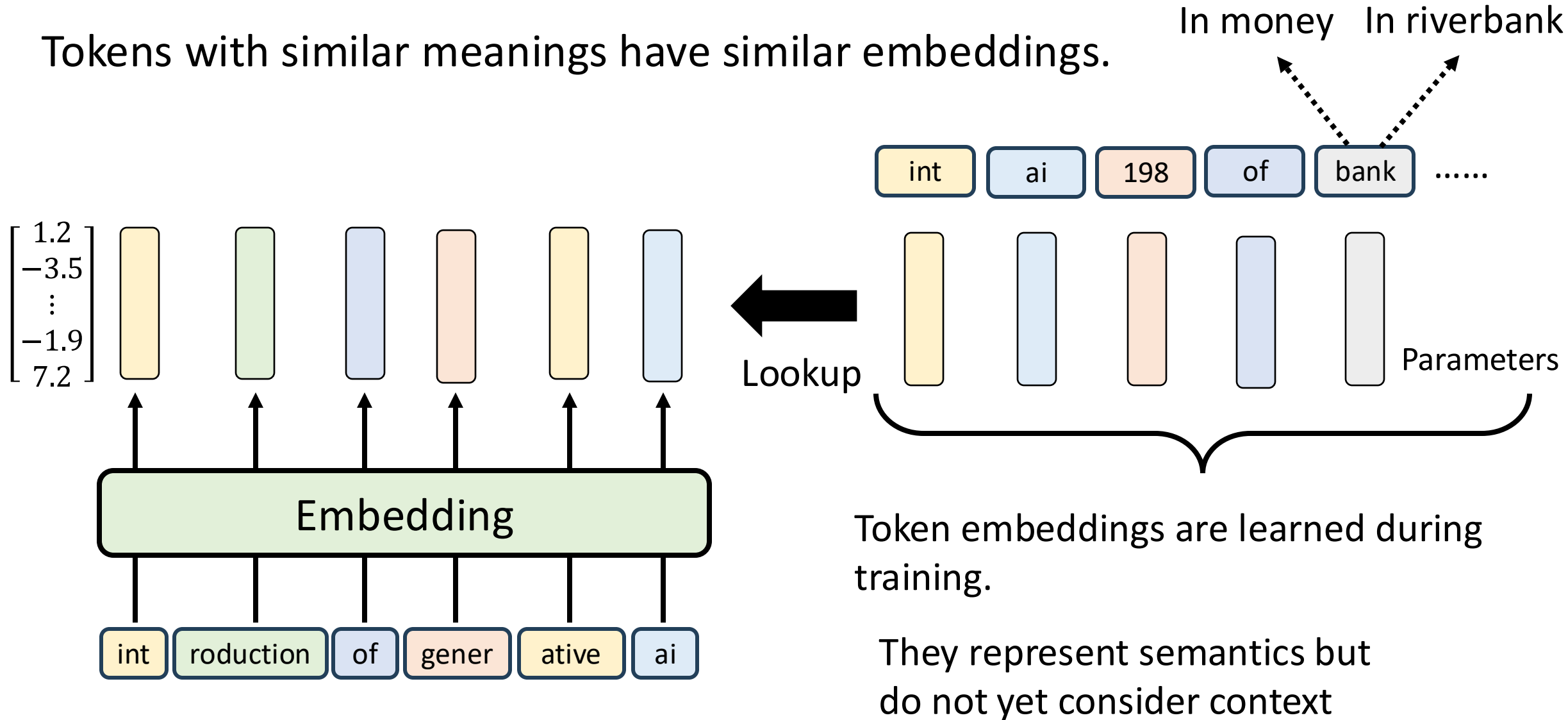
$$\begin{bmatrix} 1.2 \\ -3.5 \\ \vdots \\ -1.9 \\ 7.2 \end{bmatrix}$$


apple, cat, dot, run, jump



Step 2: Understand each token

Tokens with similar meanings have similar embeddings.



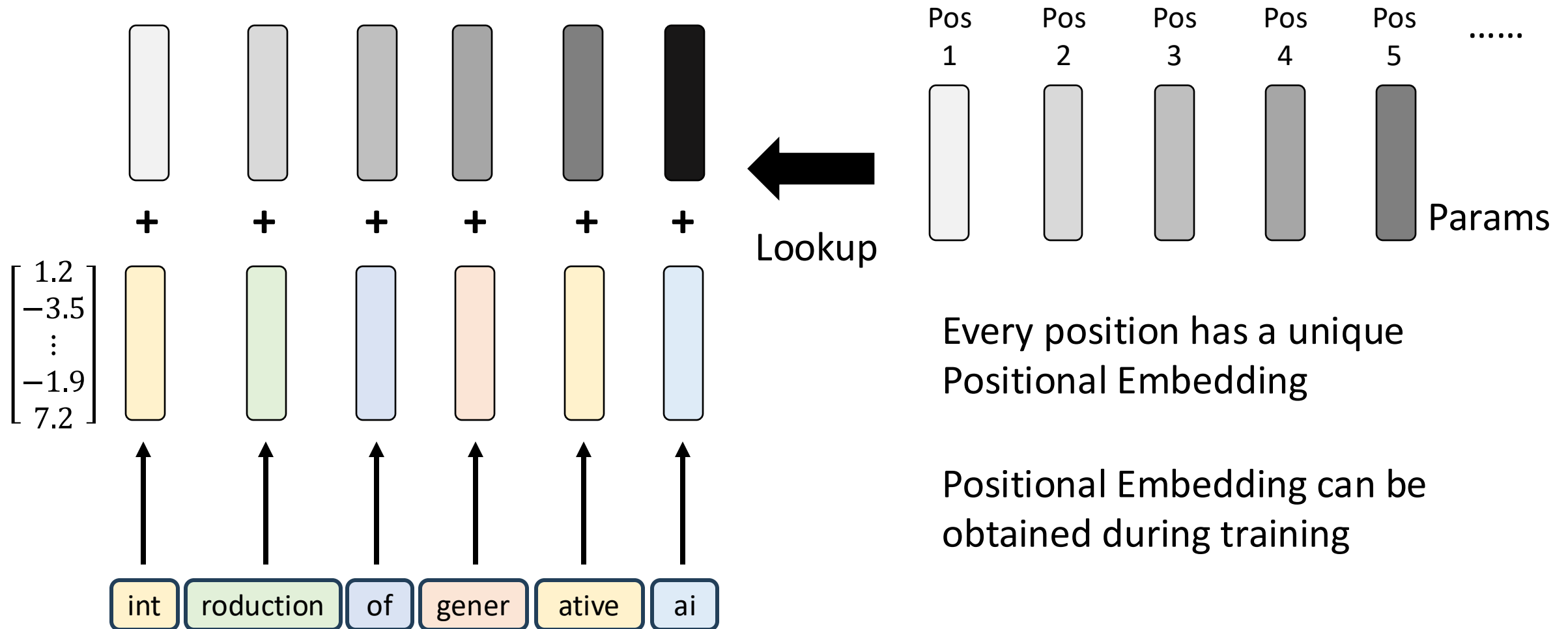
Step 2: Understand each token

- Word Embedding

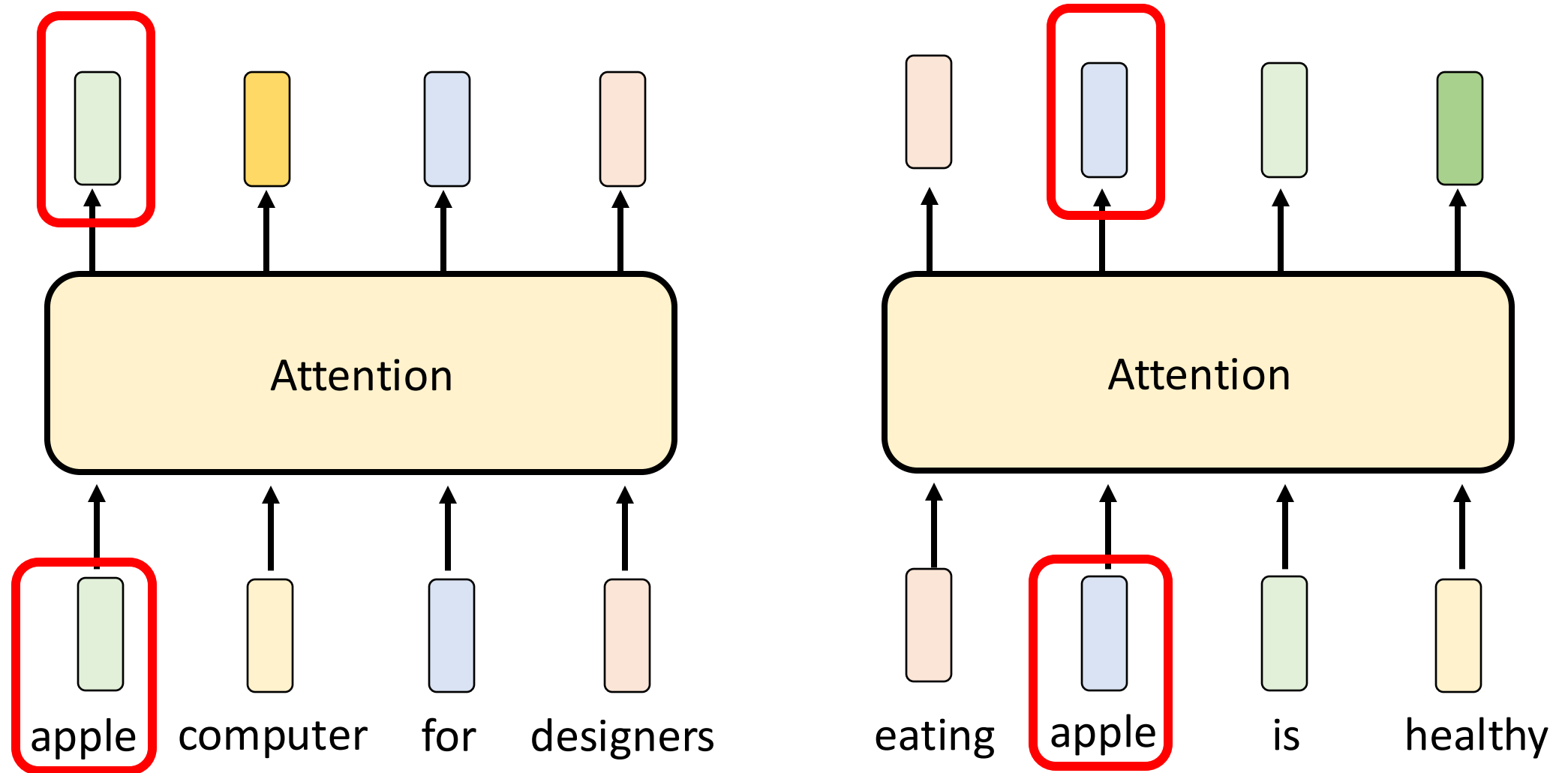


<https://youtu.be/jQTuRnjJzBU?si=QCNWQ79pkBJB70qU>

Step 2: Understand each token - Position



Step 3: Attention: understand context



Tokens exchange information with each other

Step 3: Attention: understand context

Attention Is All You Need

<https://arxiv.org/abs/1706.03762>

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

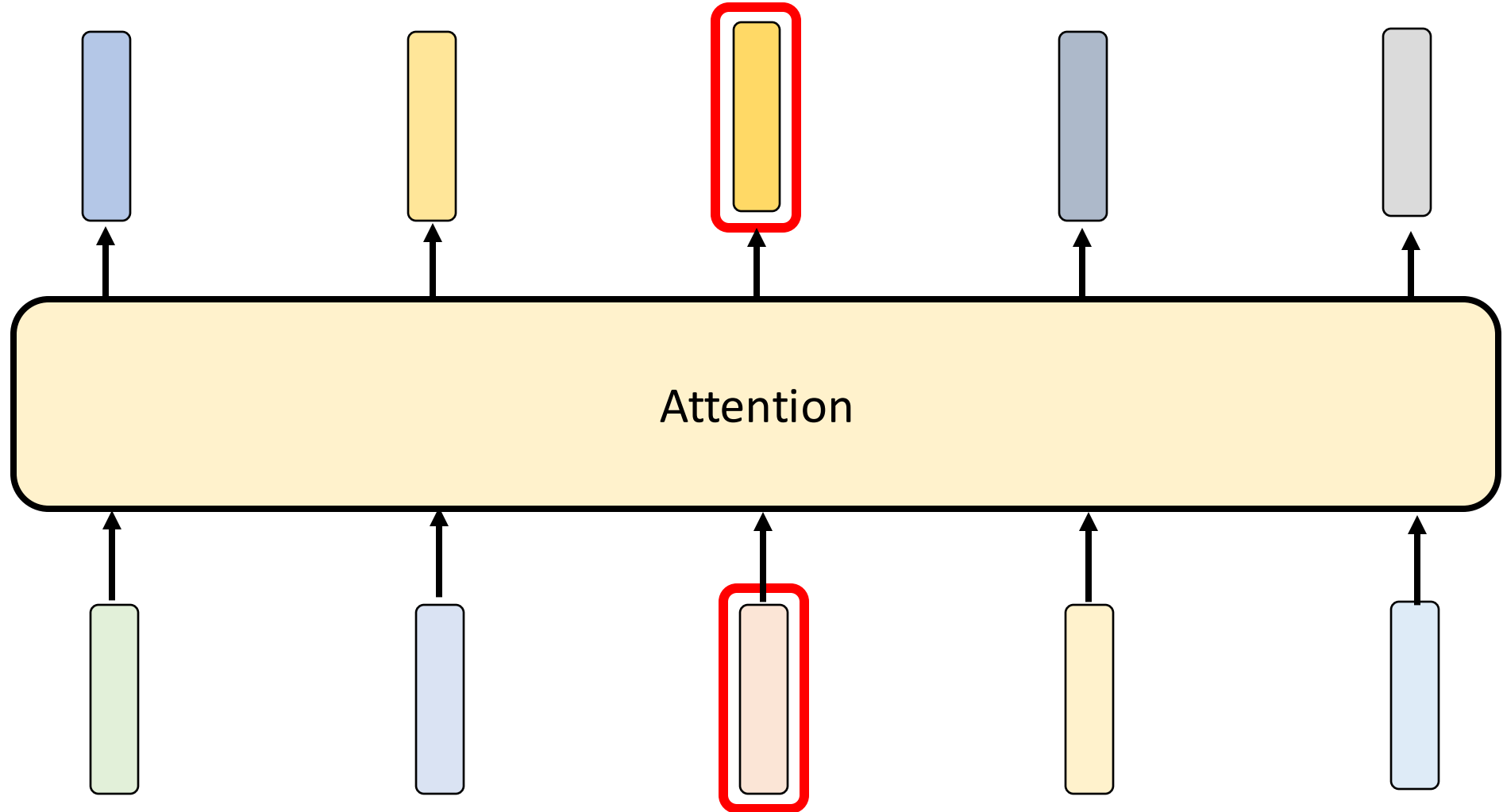
Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

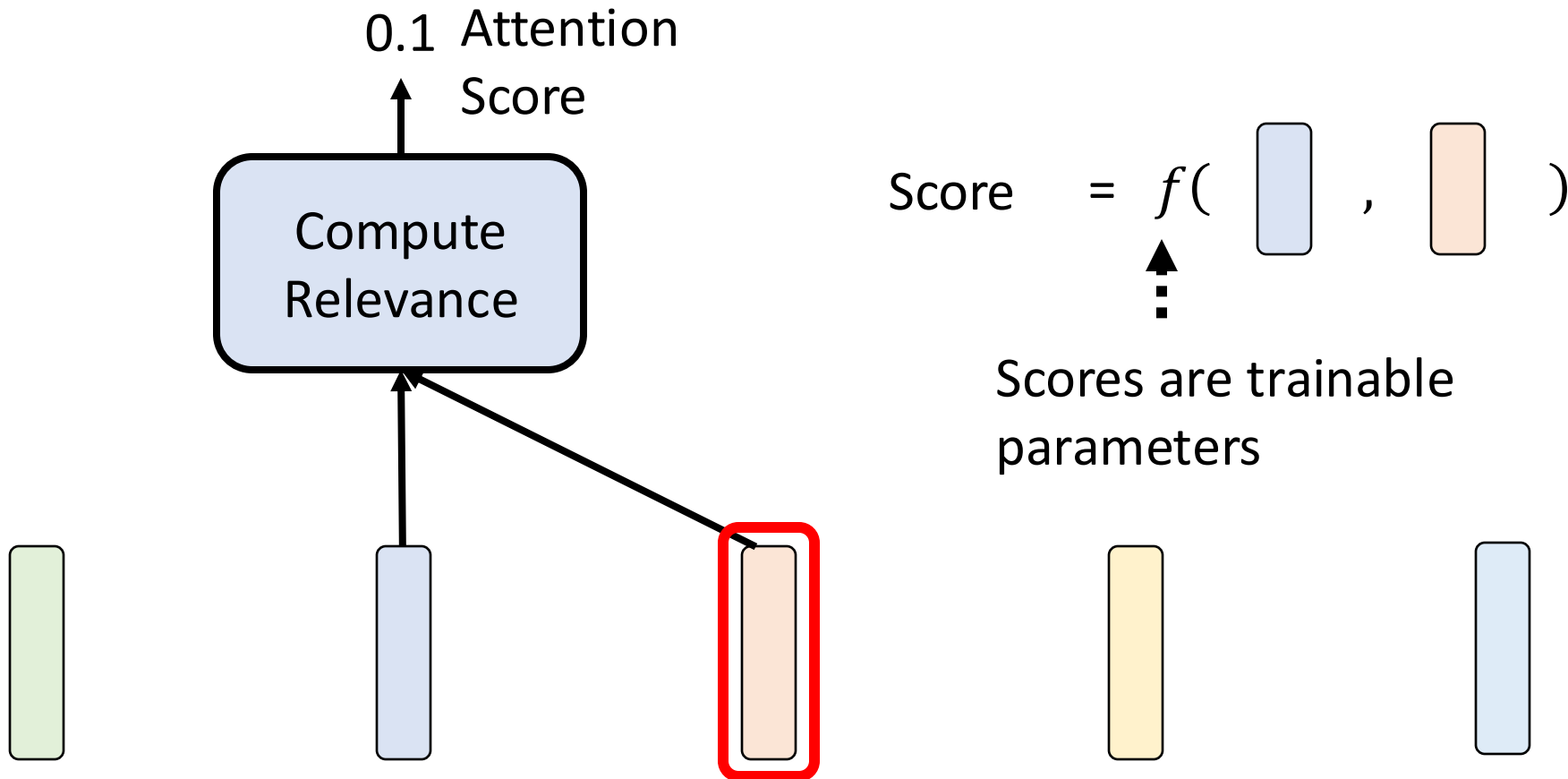
The key innovation of the Transformer paper: Recurrent Neural Networks (RNNs) are not required—Attention is sufficient.

Step 3: Attention: understand context



Step 3: Attention: understand context

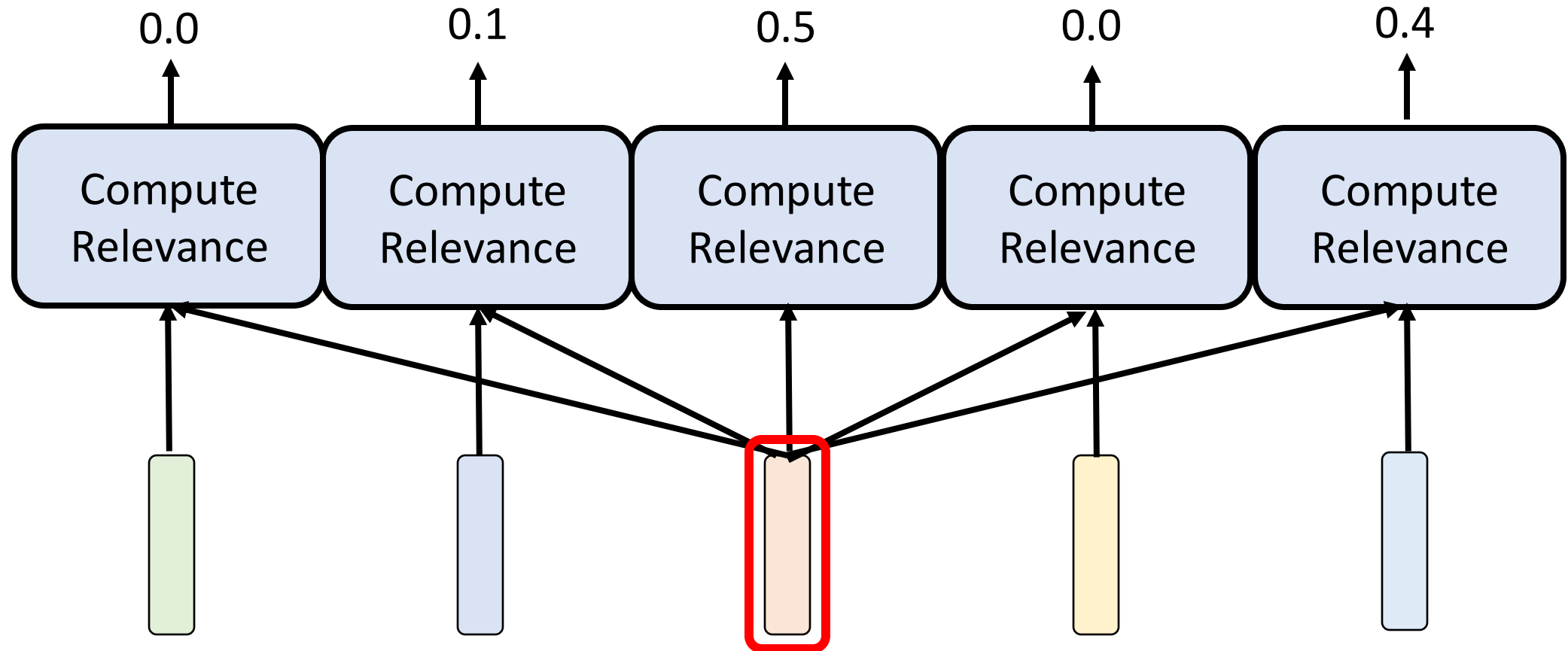
Finding relevant tokens



Step 3: Attention: understand context

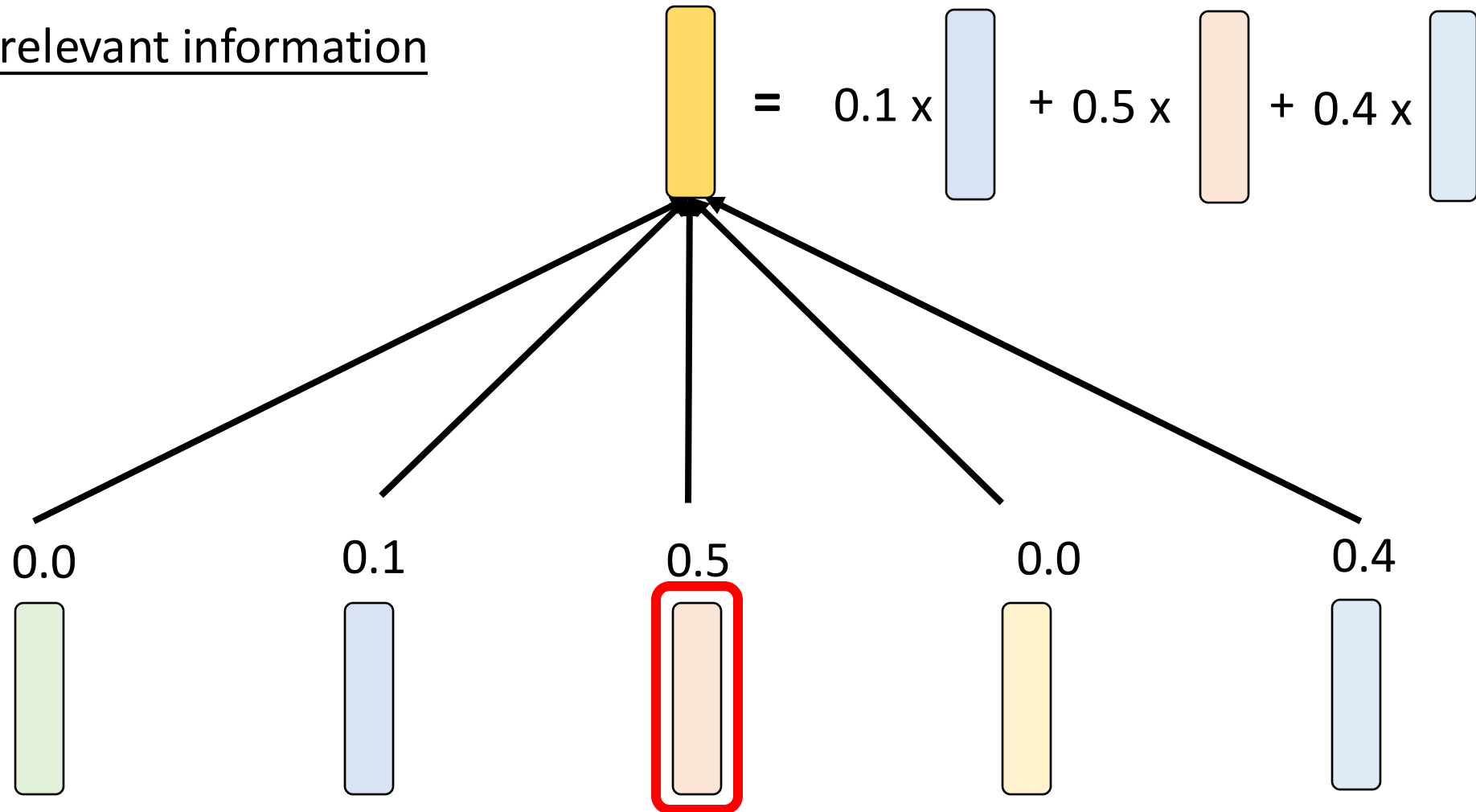
Finding relevant tokens

Attention Scores

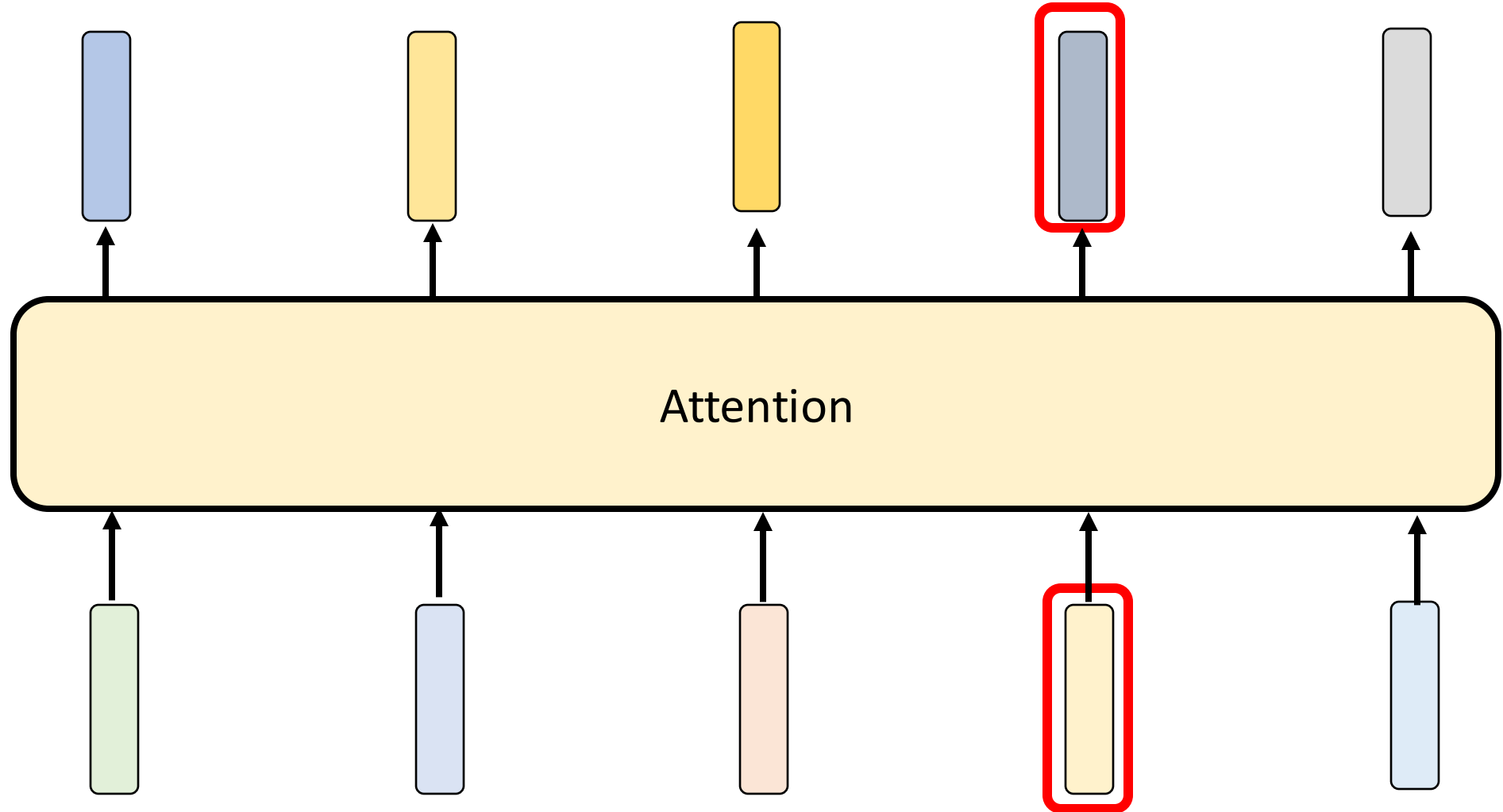


Step 3: Attention: understand context

Aggregate relevant information



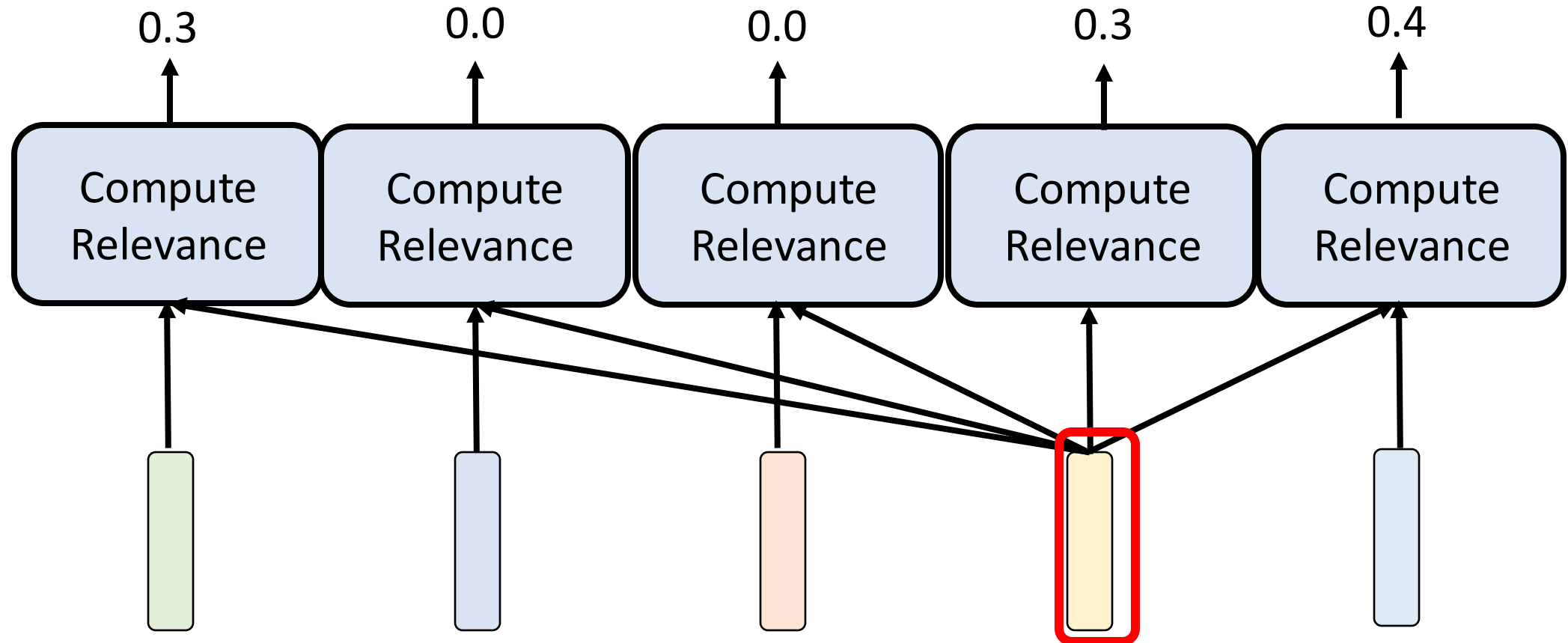
Step 3: Attention: understand context



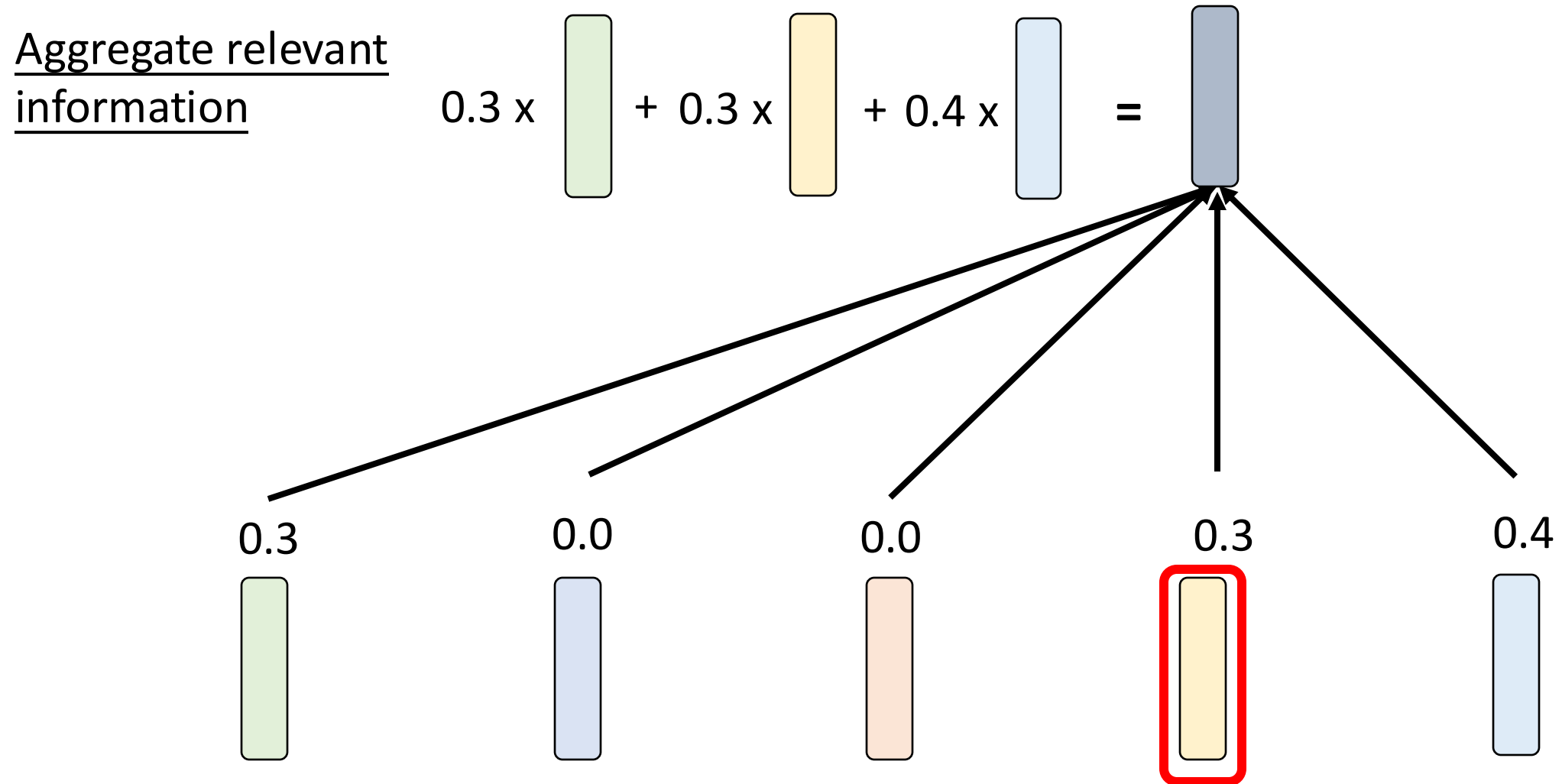
Repeat attention computations for next input token

Step 3: Attention: understand context

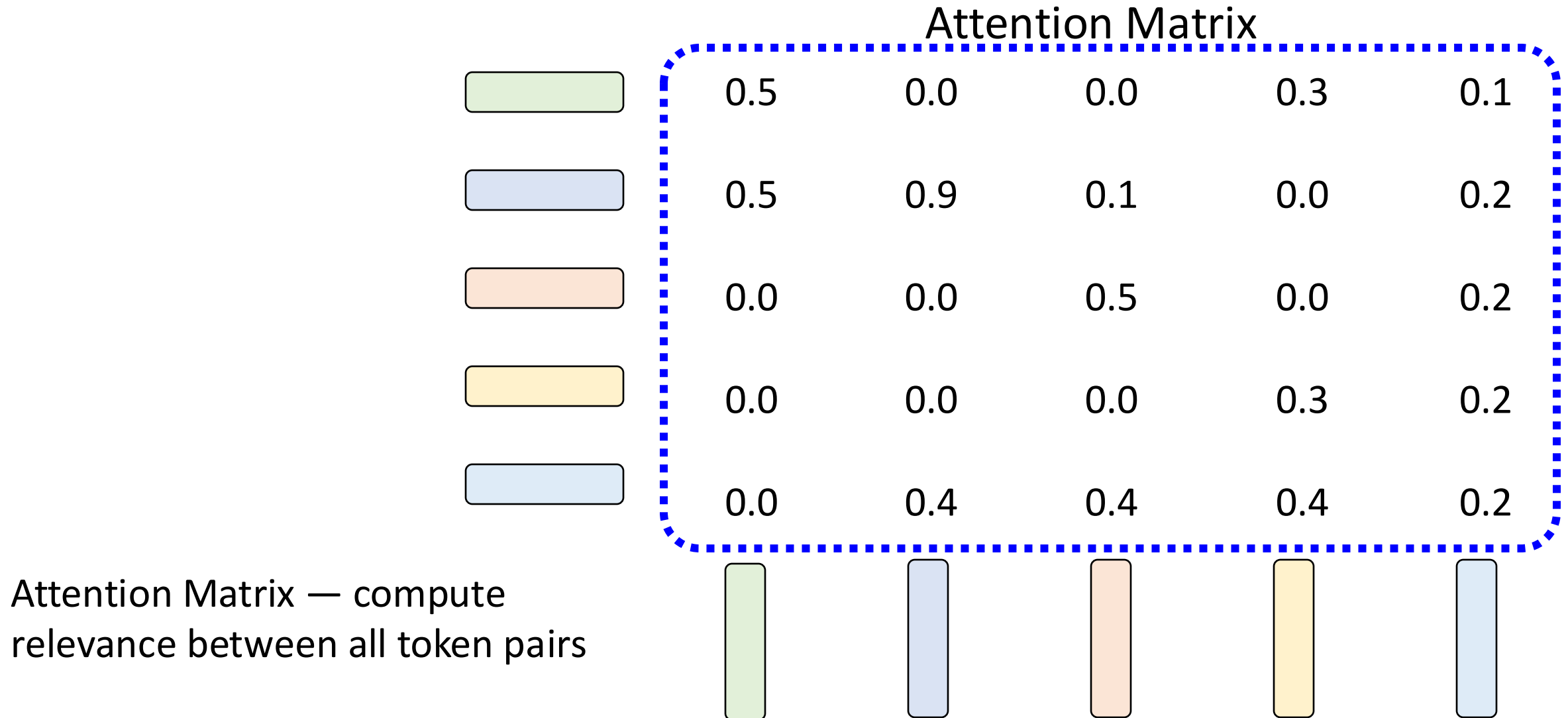
Finding relevant tokens



Step 3: Attention: understand context

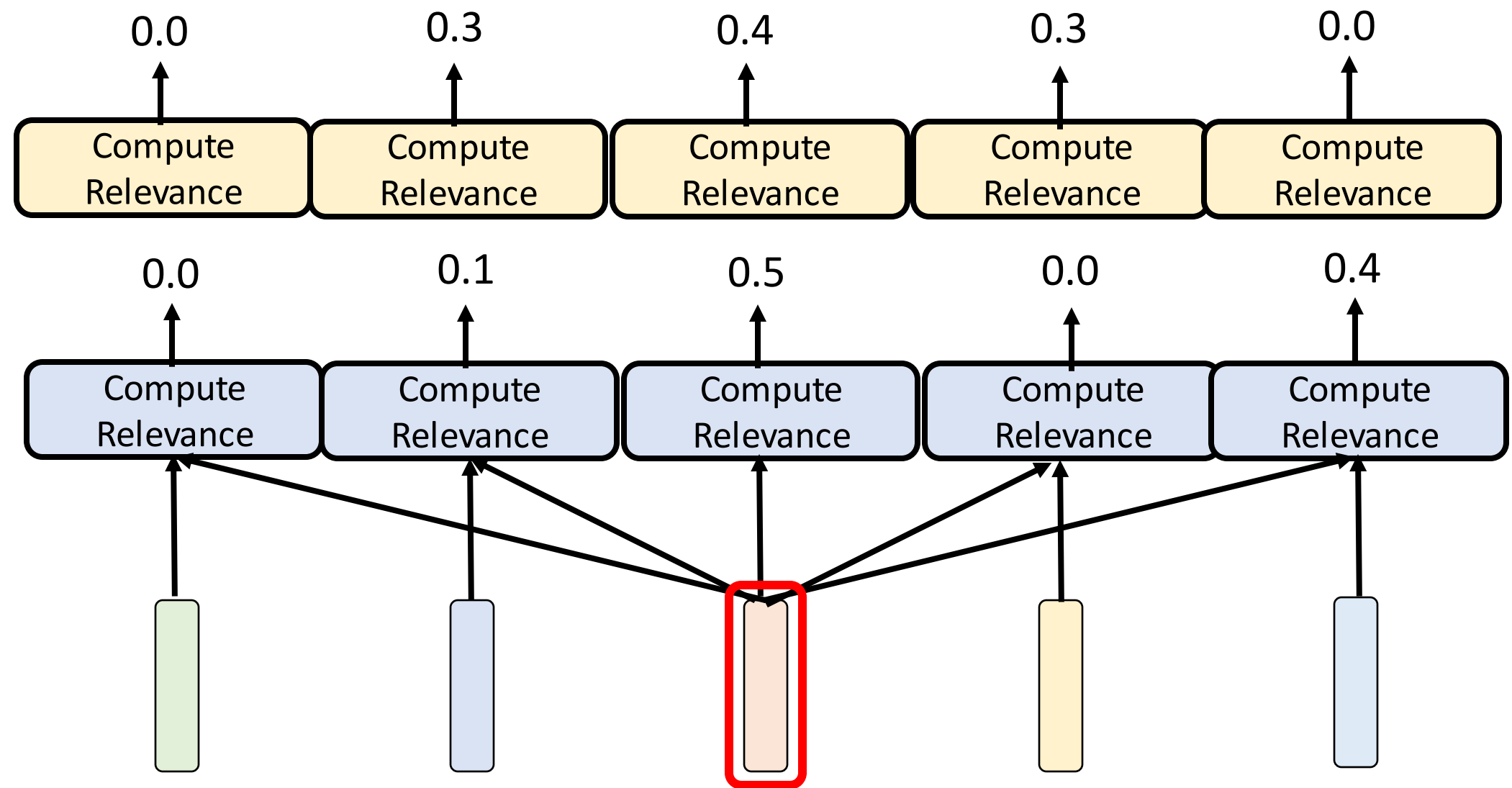


Step 3: Attention: understand context

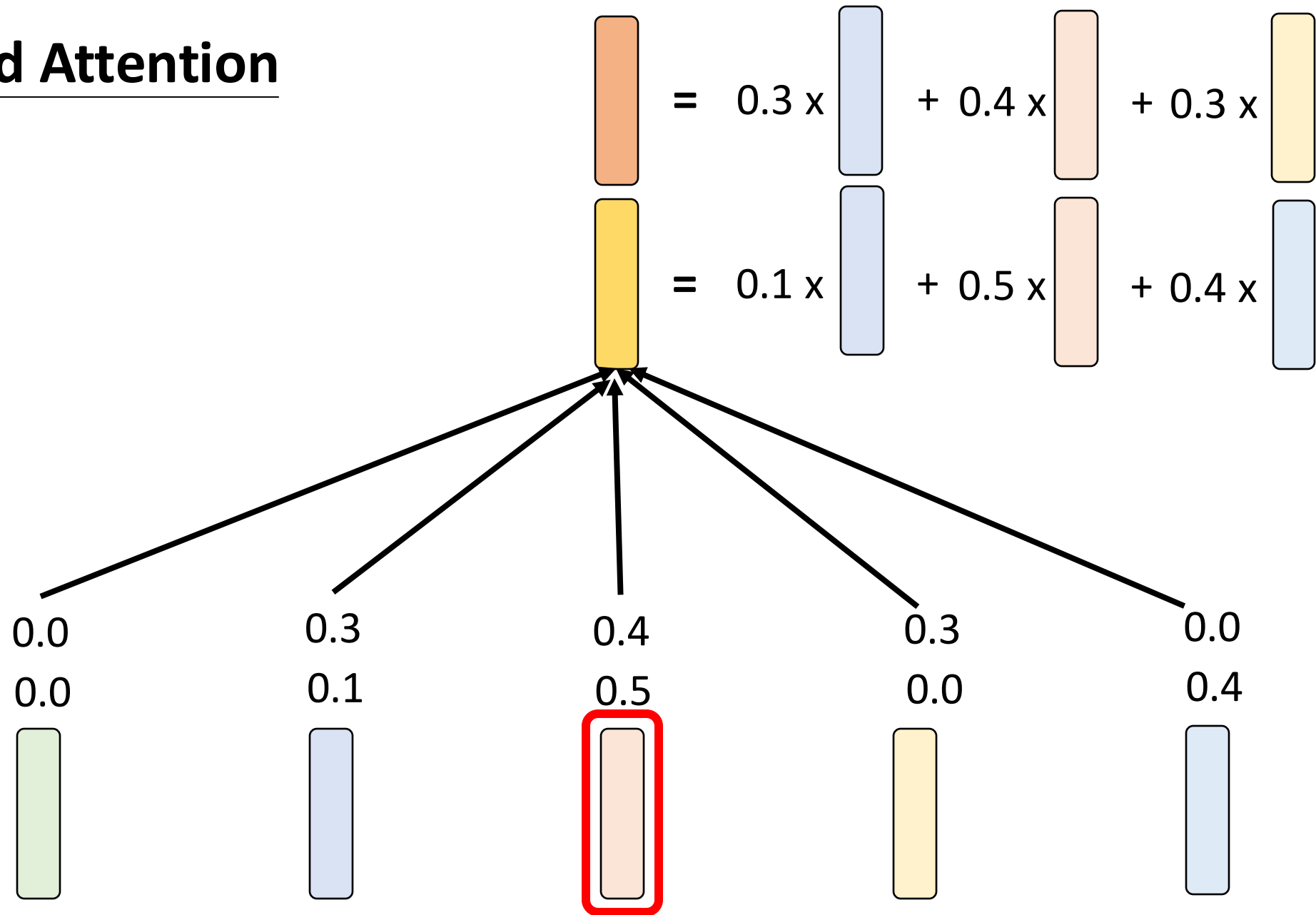


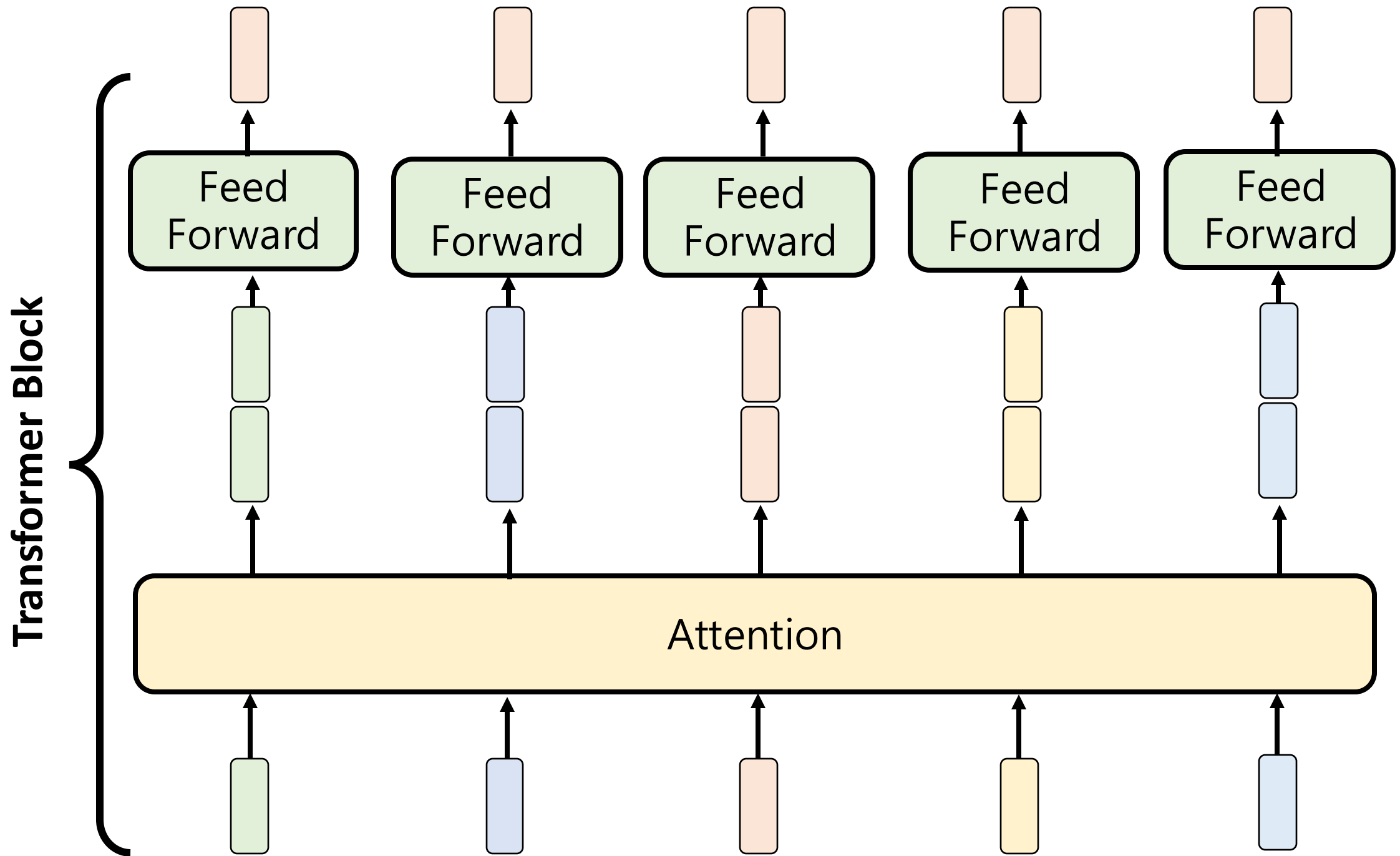
Multi-head Attention

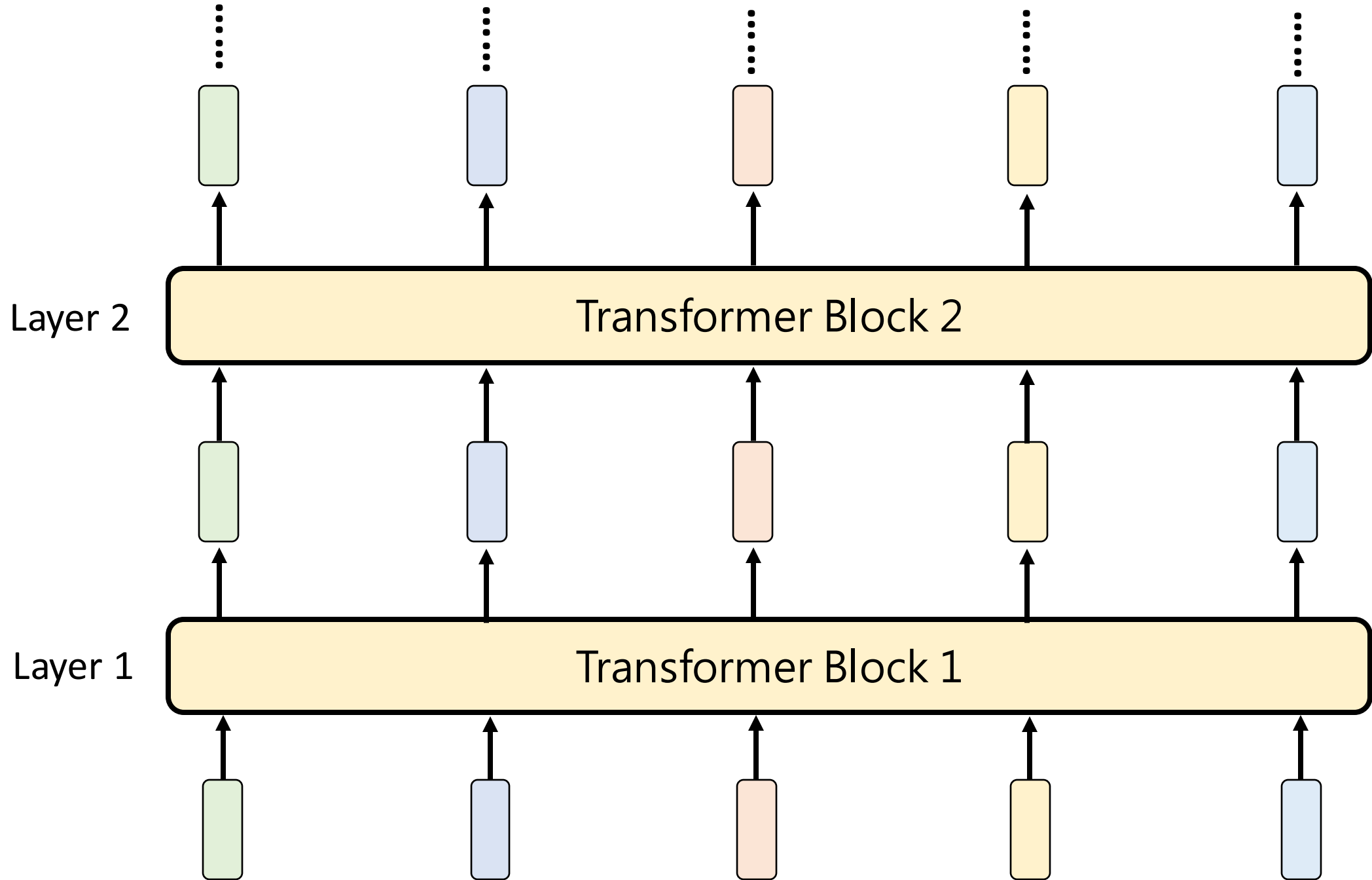
Multiple relevance types are calculated simultaneously for richer understanding

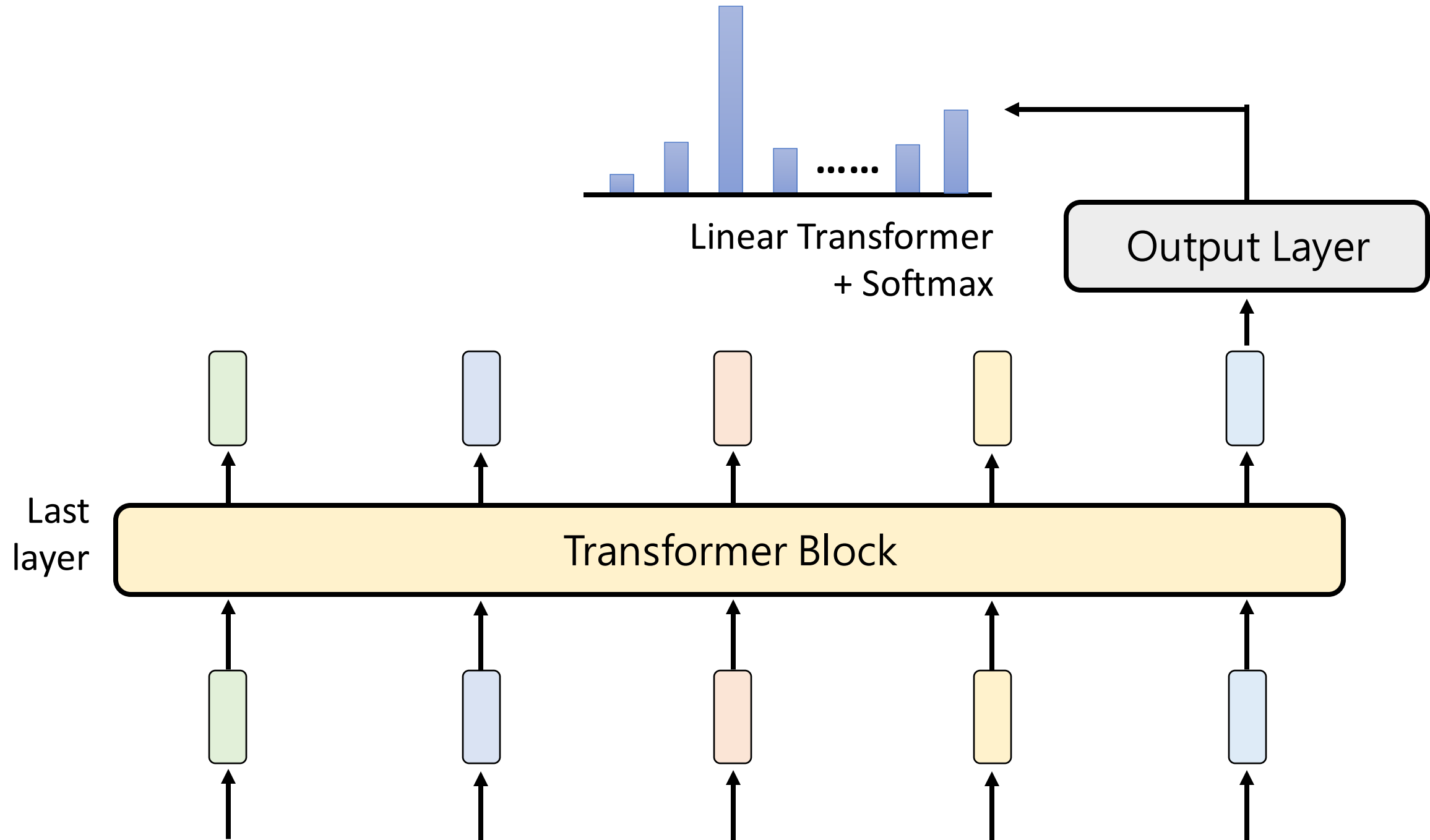


Multi-head Attention









Self-attention

<https://arxiv.org/abs/1706.03762>

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com



q : query (to match others)

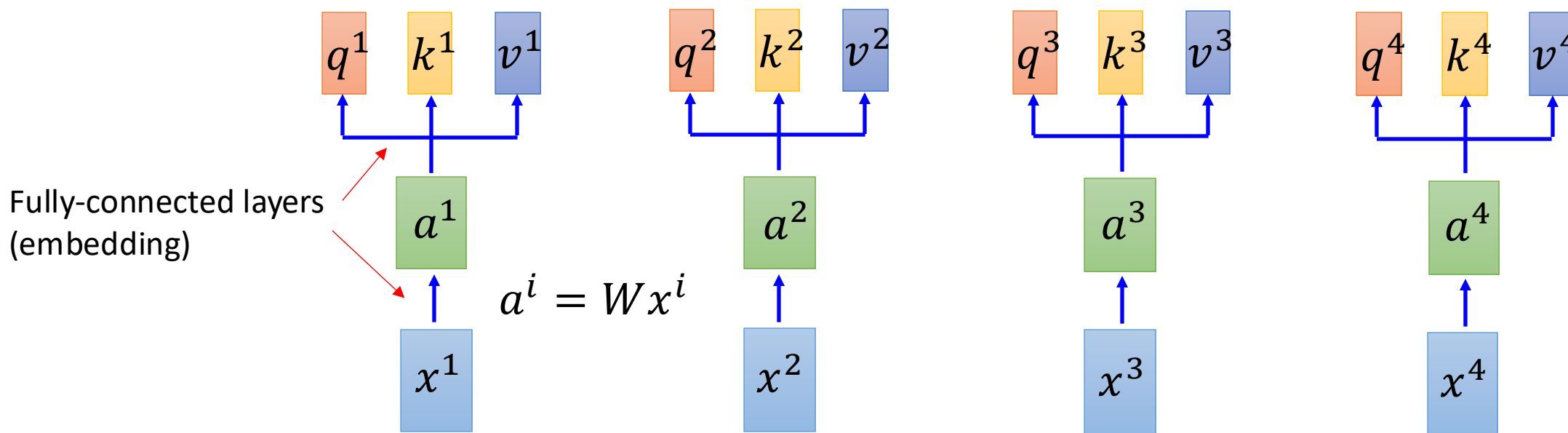
$$q^i = W^q a^i$$

k : key (to be matched)

$$k^i = W^k a^i$$

v : information to be extracted

$$v^i = W^v a^i$$



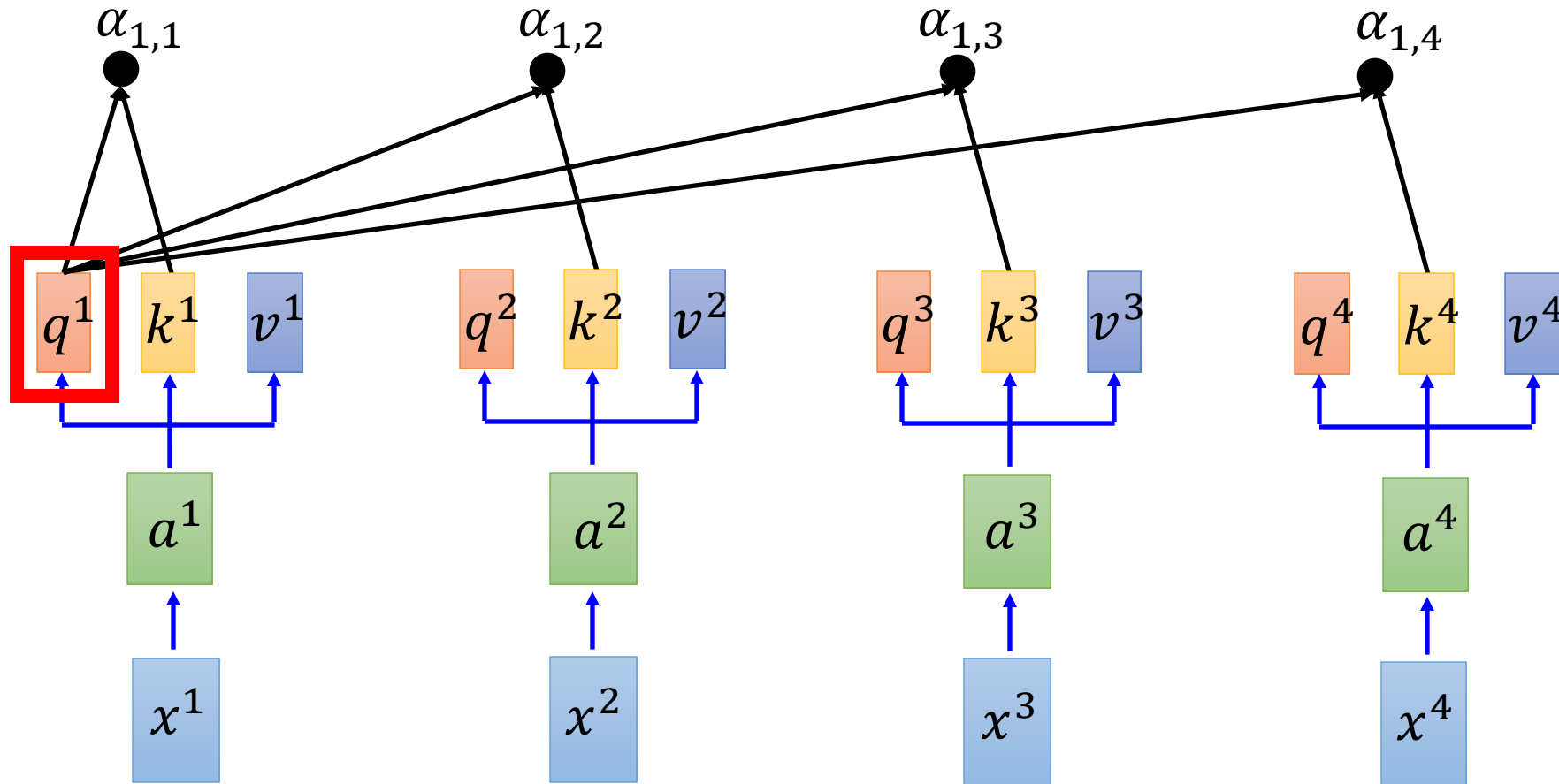
Self-attention

Use each **query** q time **key** k for attention

d is the dim of q and k

Scaled Dot-Product Attention: $\alpha_{1,i} = \underbrace{q^1 \cdot k^i}_{\text{dot product}} / \sqrt{d}$

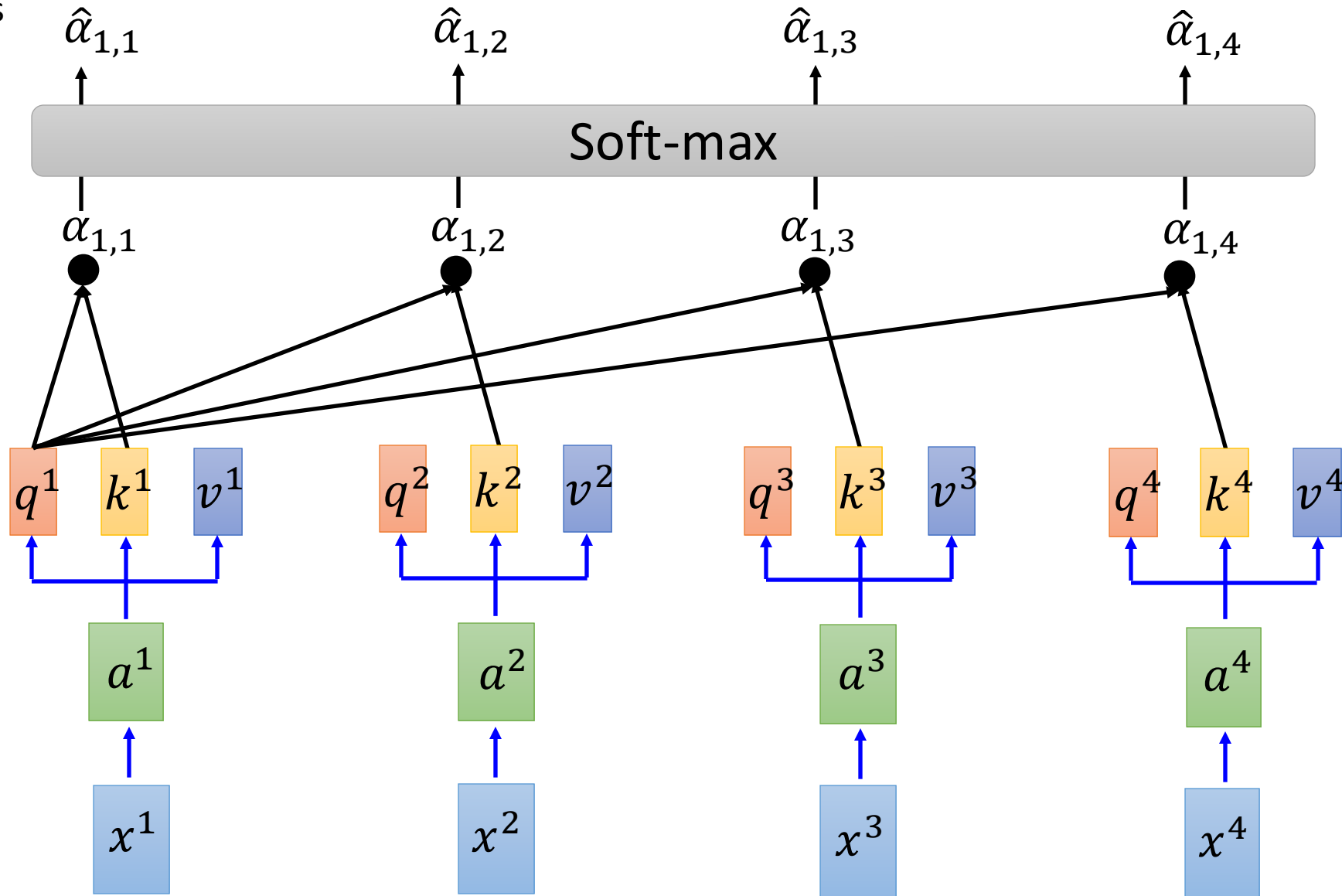
dot product



Self-attention

$$\hat{\alpha}_{1,i} = \exp(\alpha_{1,i}) / \sum_j \exp(\alpha_{1,j})$$

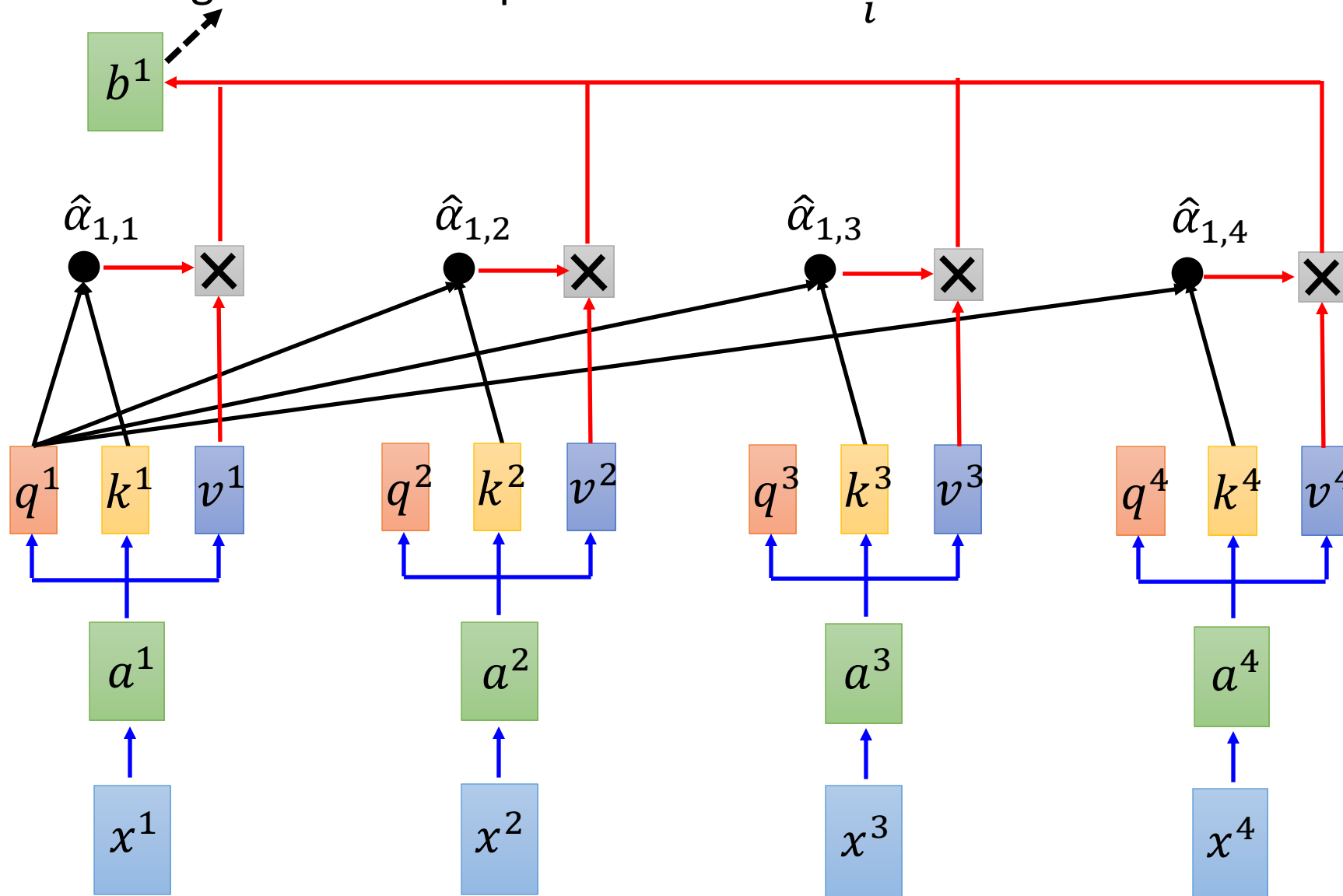
Probabilities
sum up to 1



Self-attention

$$b^1 = \sum_i \hat{\alpha}_{1,i} v^i$$

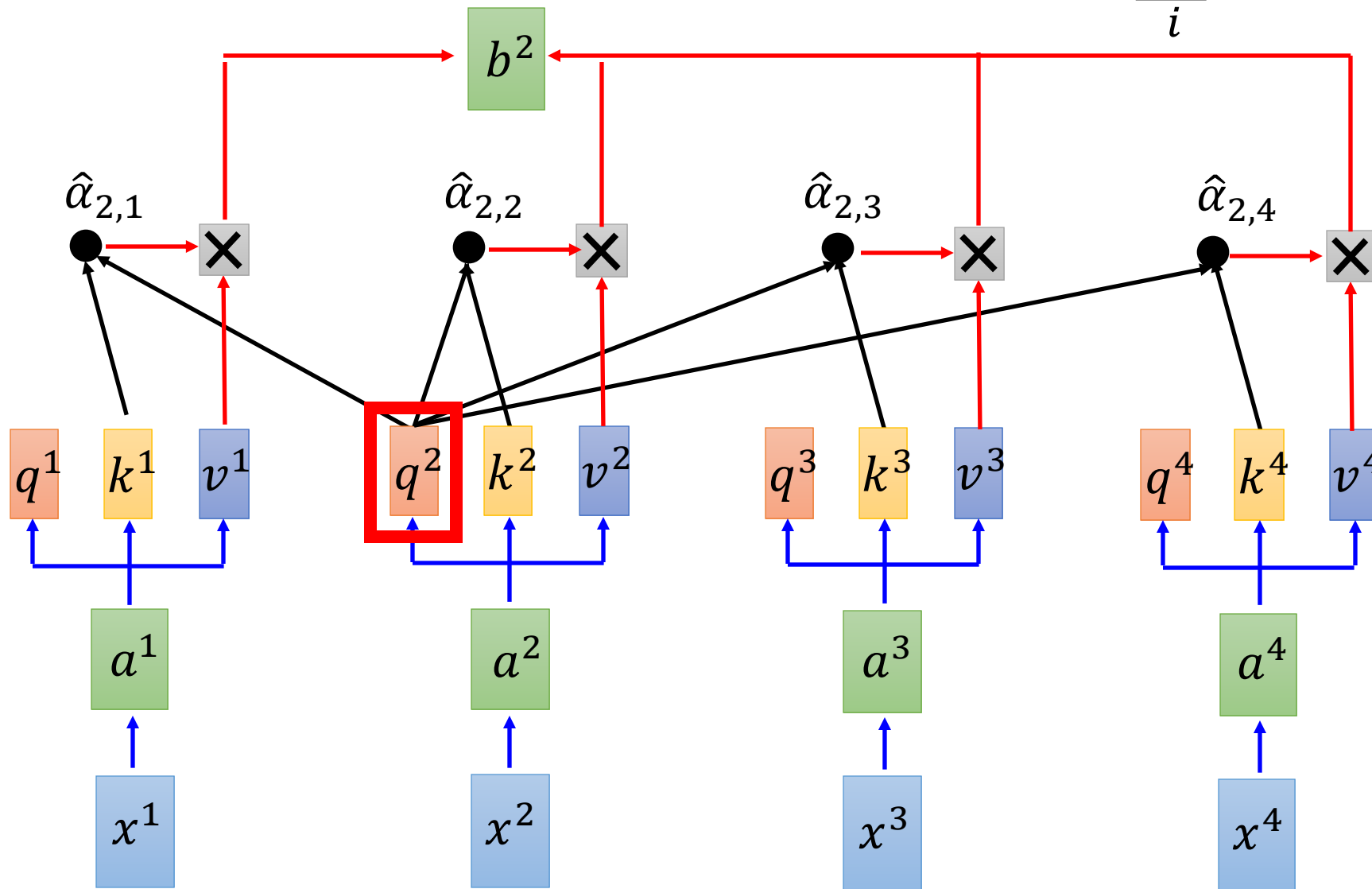
Considering the whole sequence



Self-attention

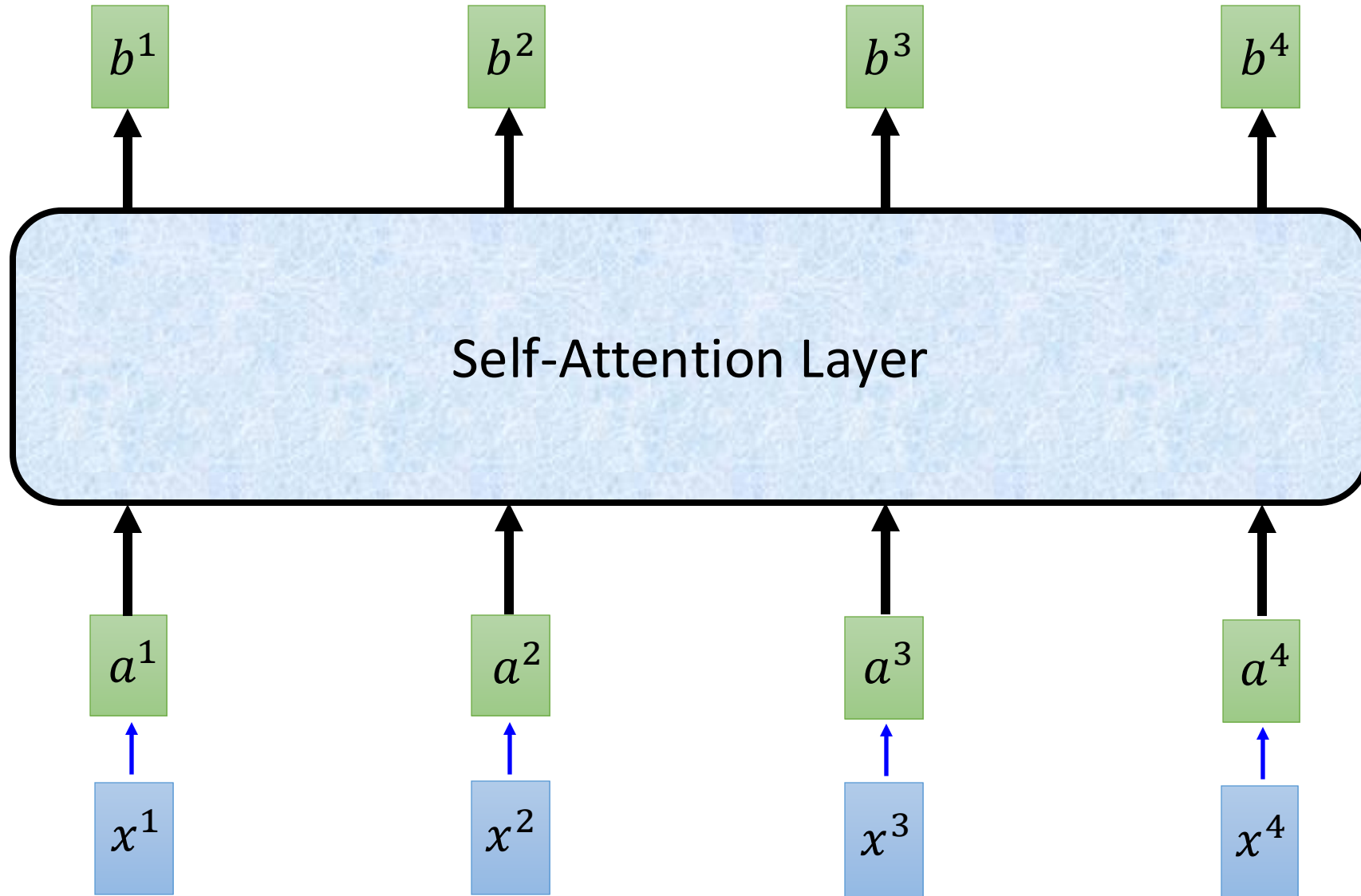
Use each **query** q time **key** k for attention

$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$



Self-attention

b^1, b^2, b^3, b^4 can be parallelly computed.



Self-attention

$$q^i = W^q a^i$$

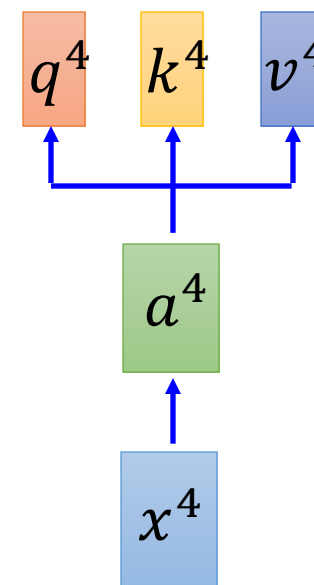
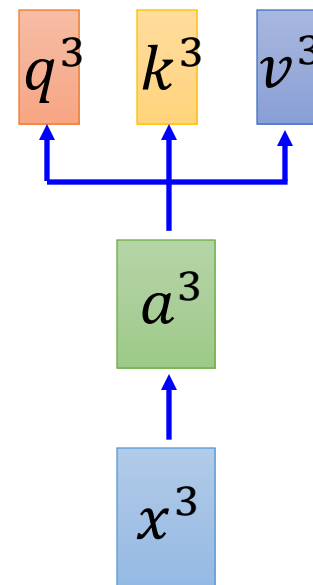
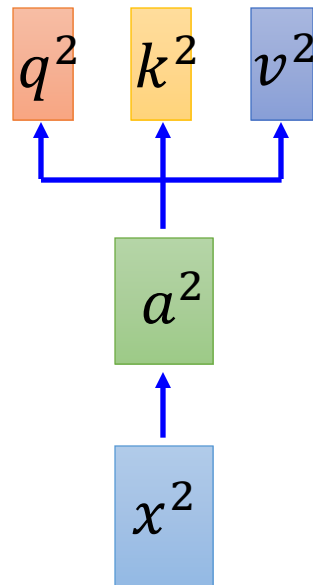
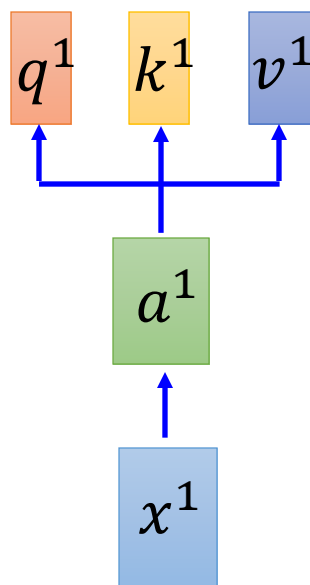
$$k^i = W^k a^i$$

$$v^i = W^v a^i$$

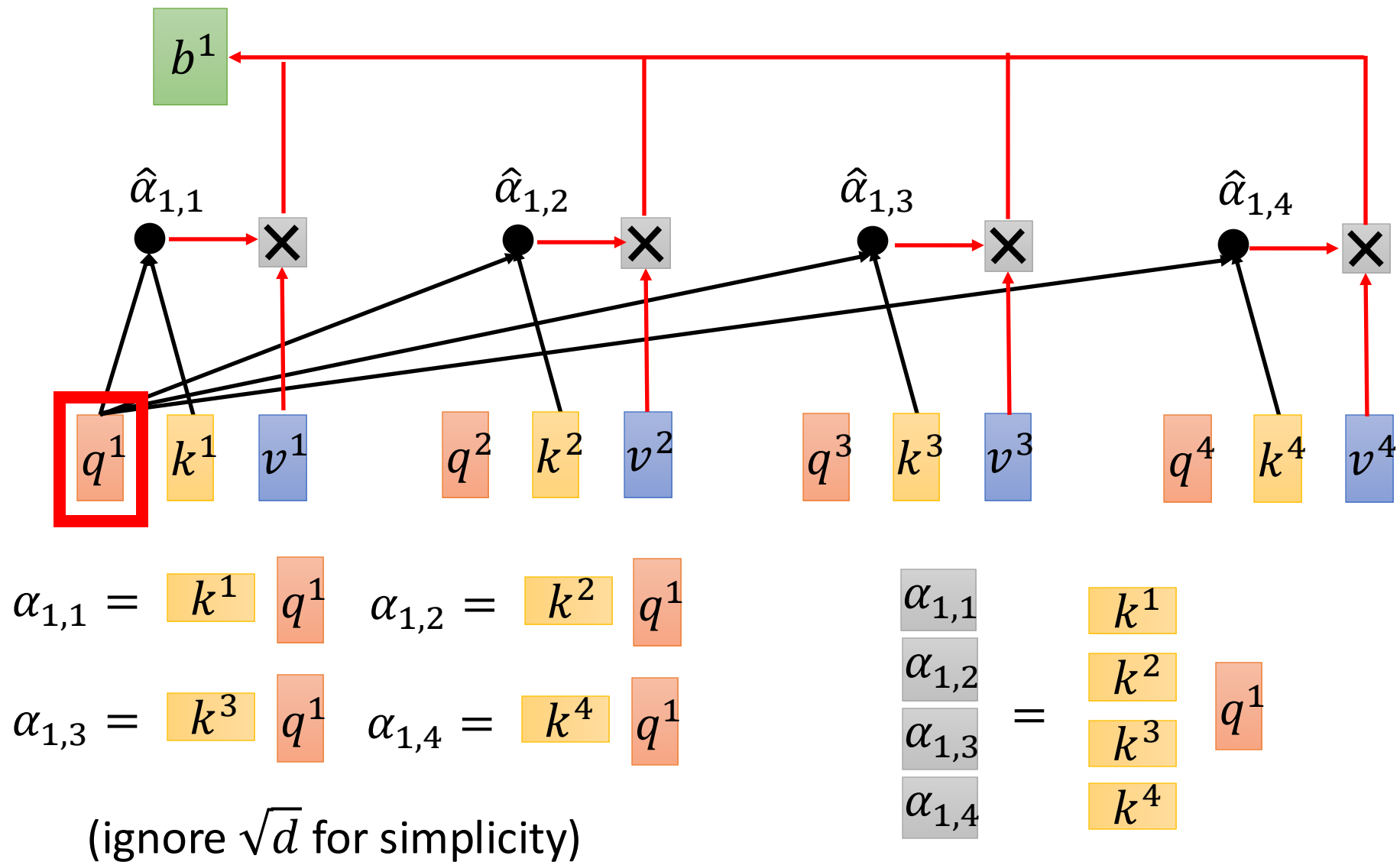
$$\begin{array}{c} q^1 q^2 q^3 q^4 \\ Q \end{array} = \begin{array}{c} W^q \\ \text{I} \end{array} \begin{array}{c} a^1 a^2 a^3 a^4 \\ \text{I} \end{array}$$

$$\begin{array}{c} k^1 k^2 k^3 k^4 \\ K \end{array} = \begin{array}{c} W^k \\ \text{I} \end{array} \begin{array}{c} a^1 a^2 a^3 a^4 \\ \text{I} \end{array}$$

$$\begin{array}{c} v^1 v^2 v^3 v^4 \\ V \end{array} = \begin{array}{c} W^v \\ \text{I} \end{array} \begin{array}{c} a^1 a^2 a^3 a^4 \\ \text{I} \end{array}$$

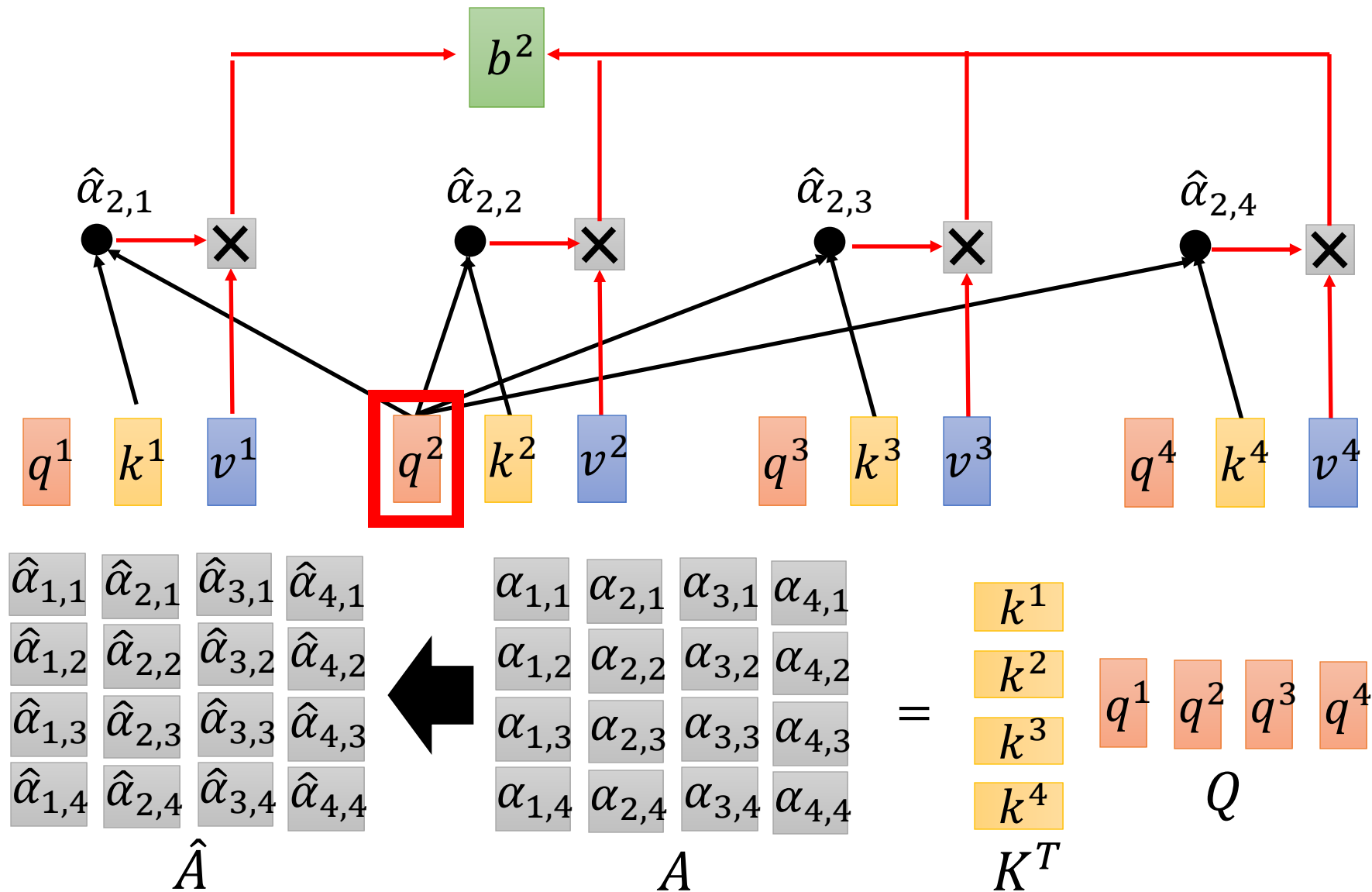


Self-attention



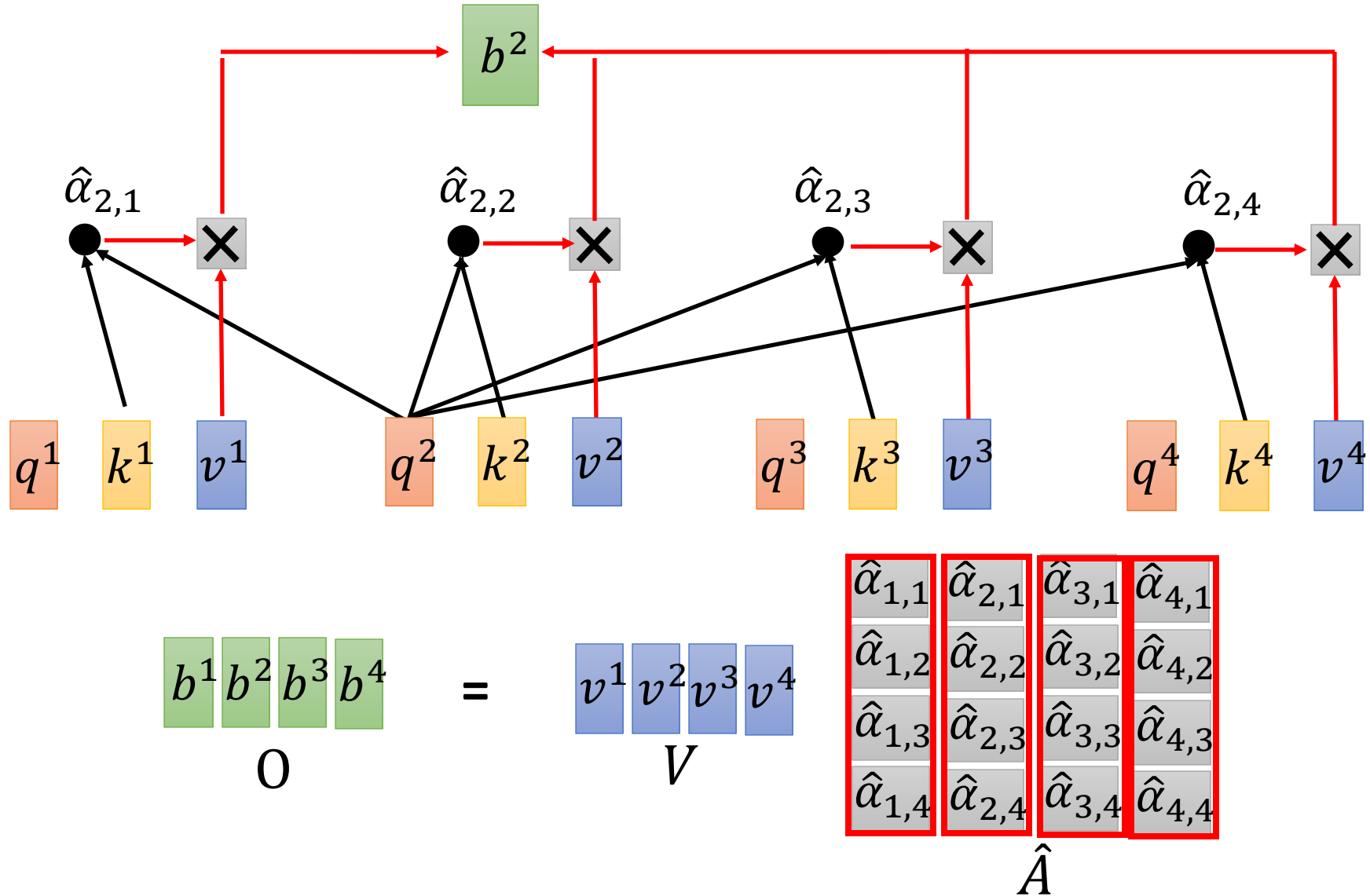
Self-attention

$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$

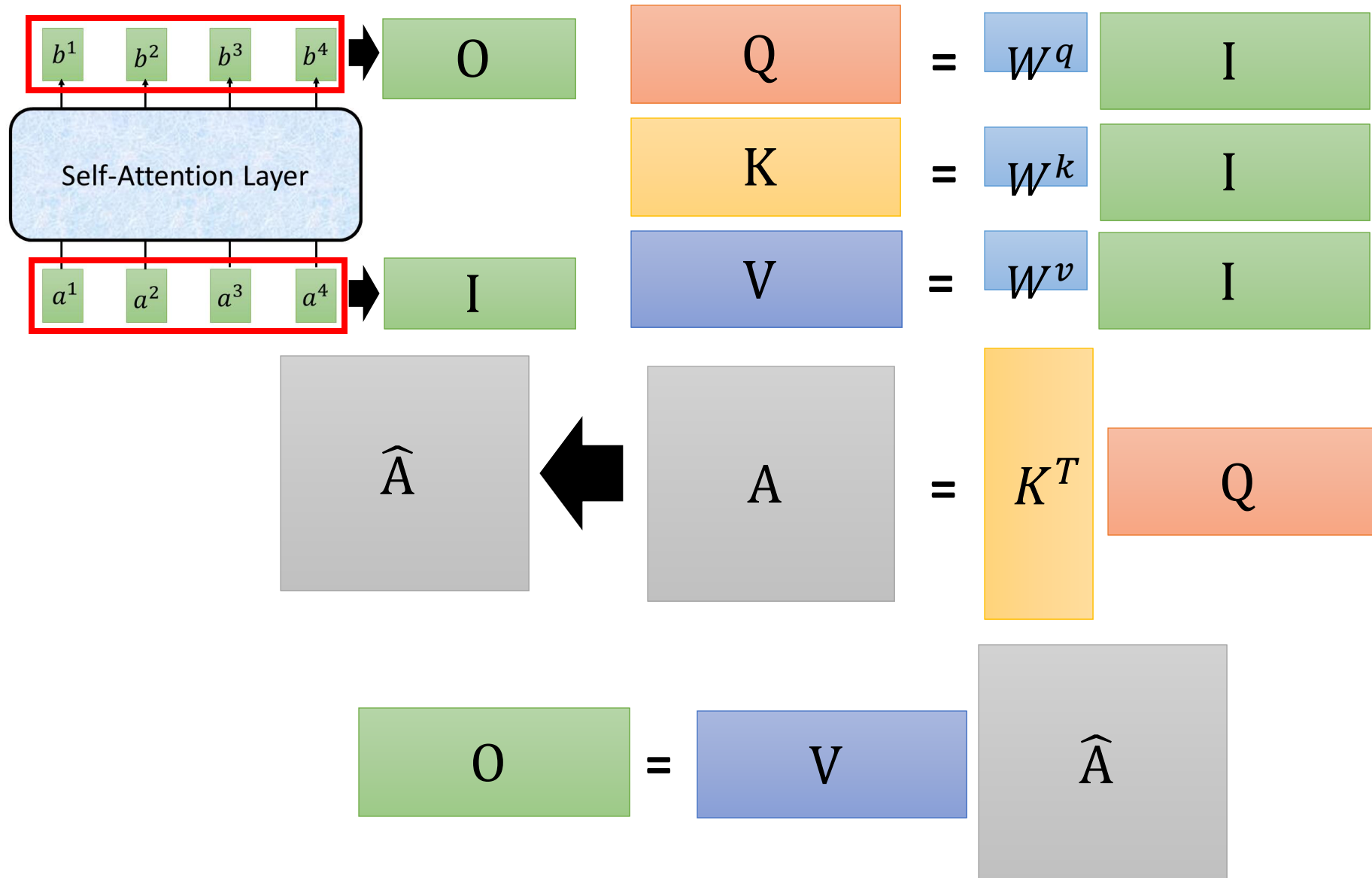


Self-attention

$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$



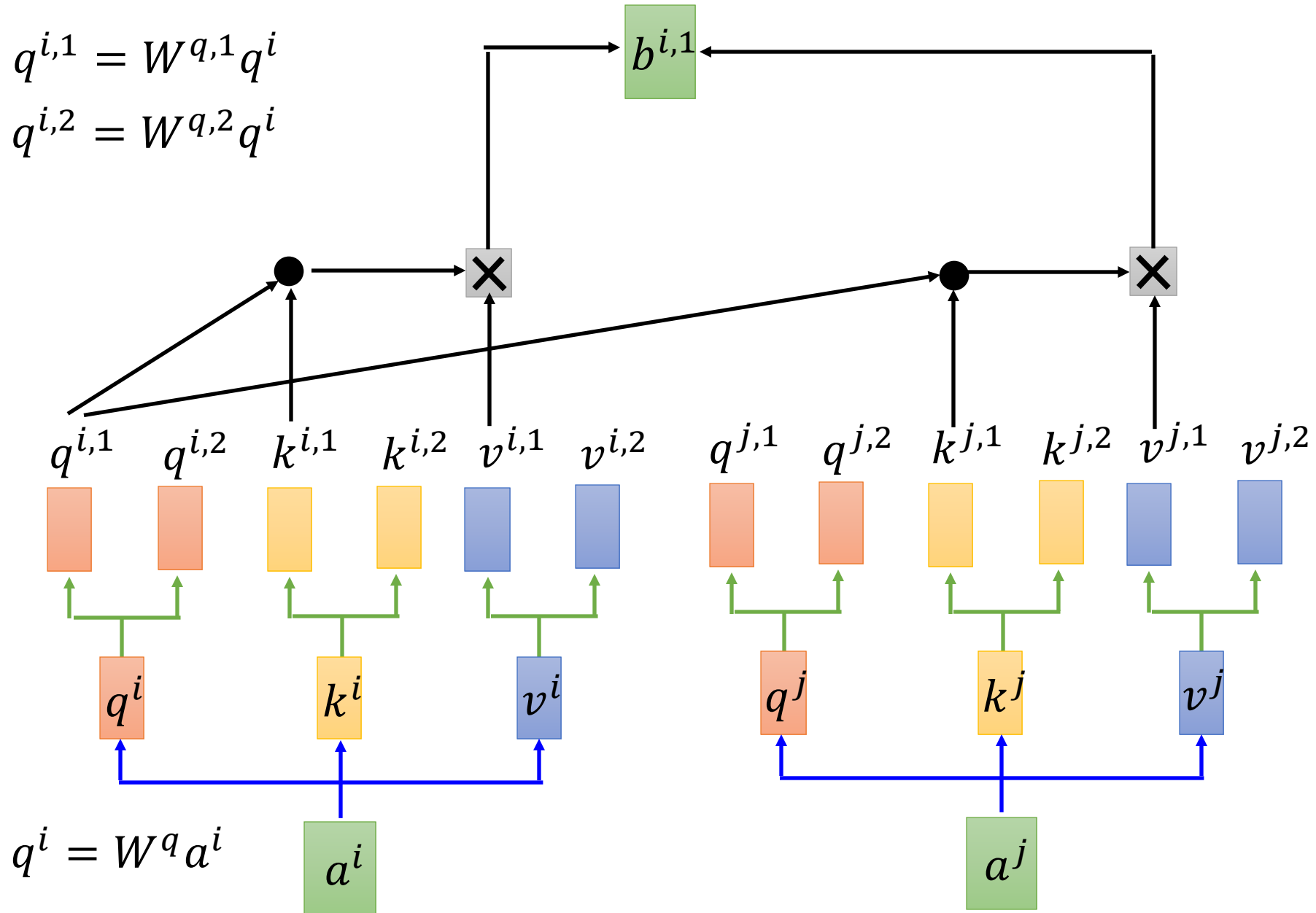
Self-attention



Everything is matrix multiplication, can be accelerated by GPU

Multi-head Self-attention

(2 heads as example)

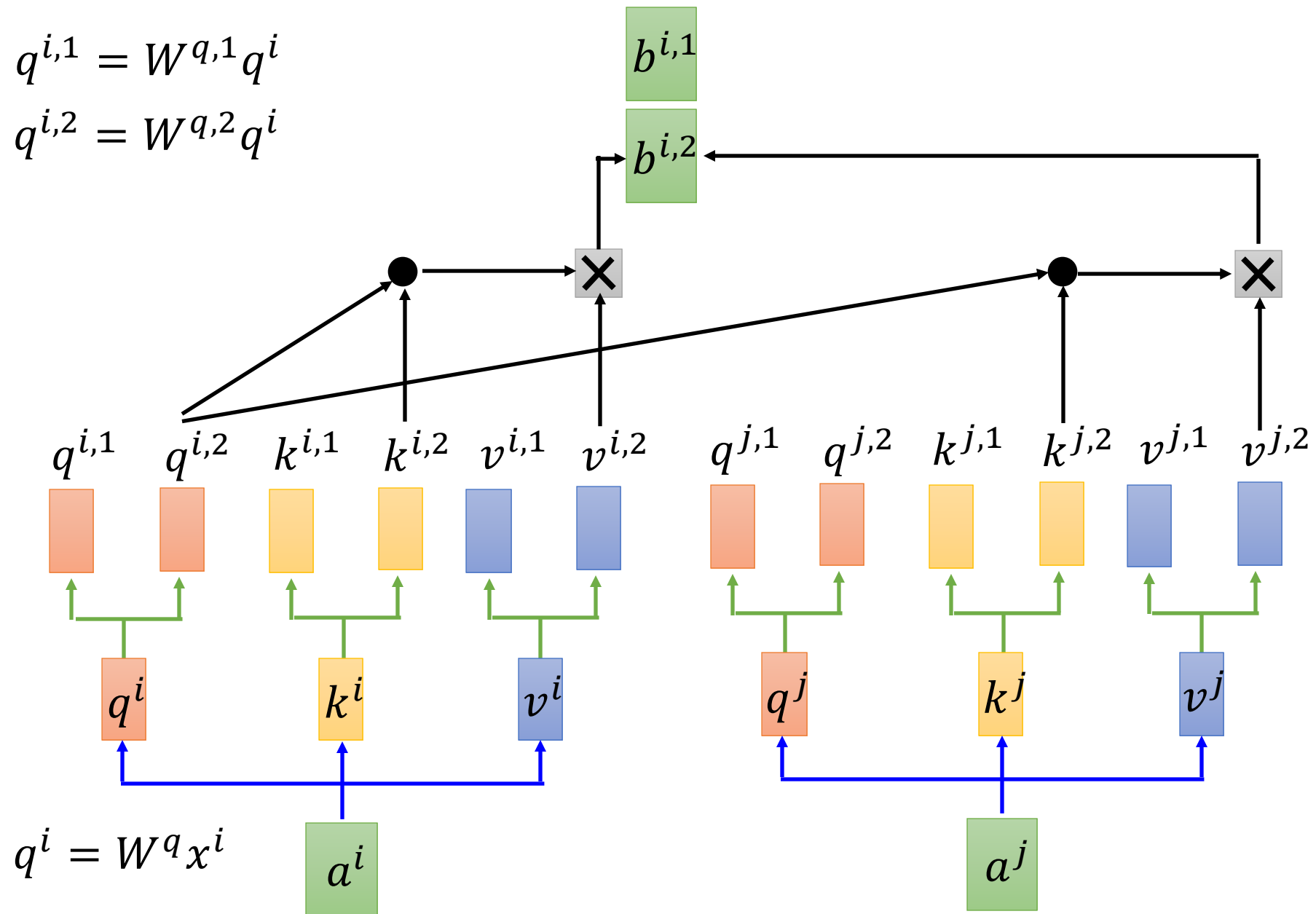


Multi-head Self-attention

(2 heads as example)

$$q^{i,1} = W^{q,1} q^i$$

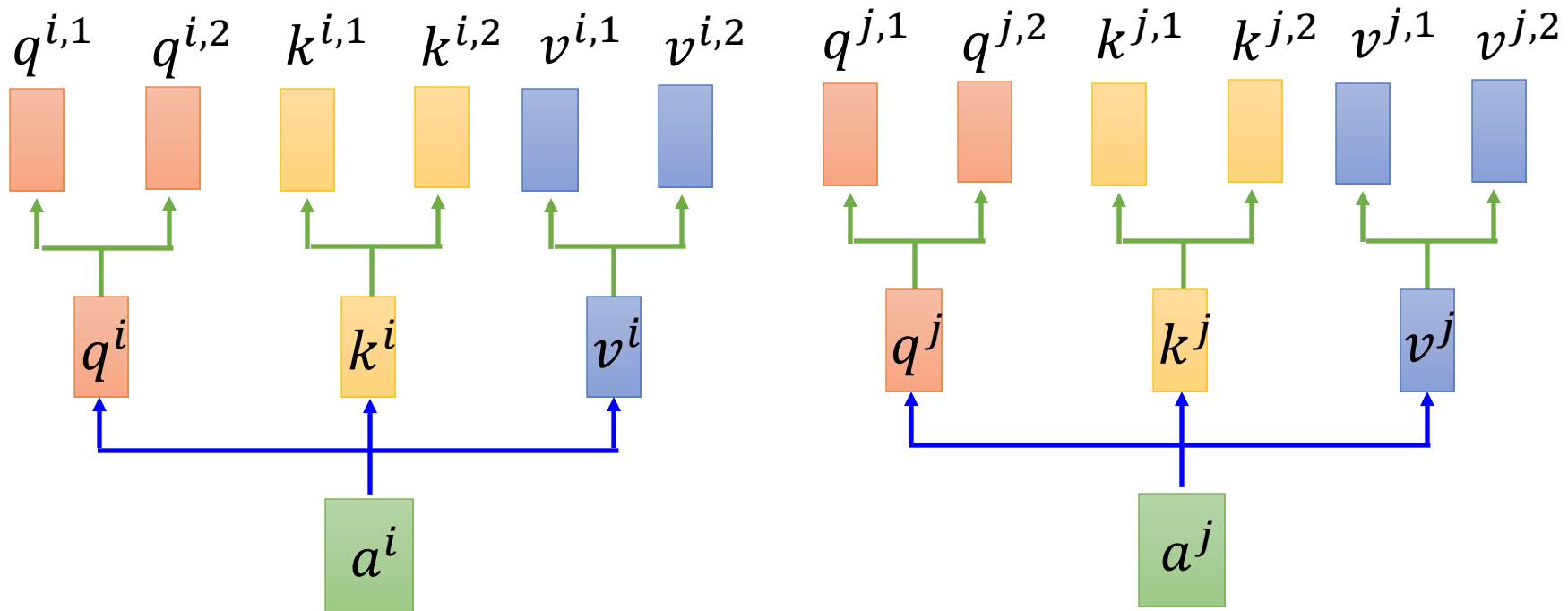
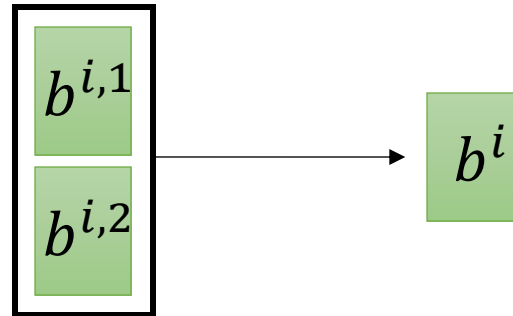
$$q^{i,2} = W^{q,2} q^i$$



Multi-head Self-attention

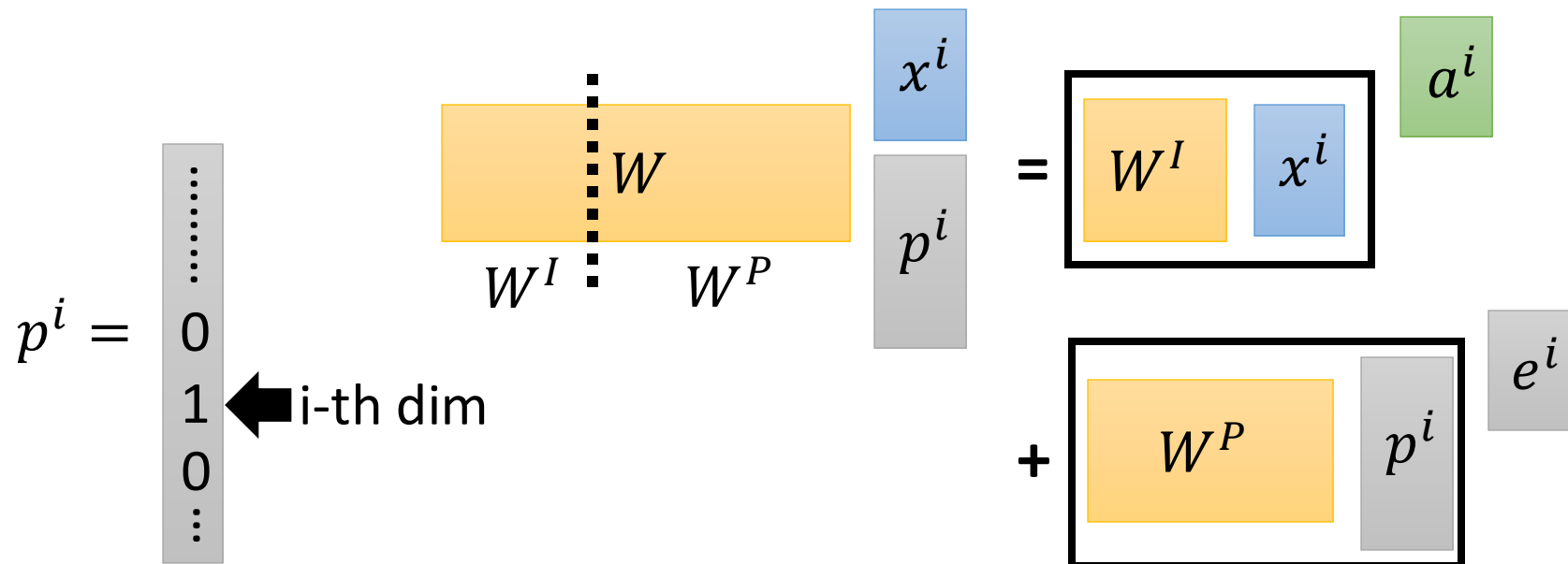
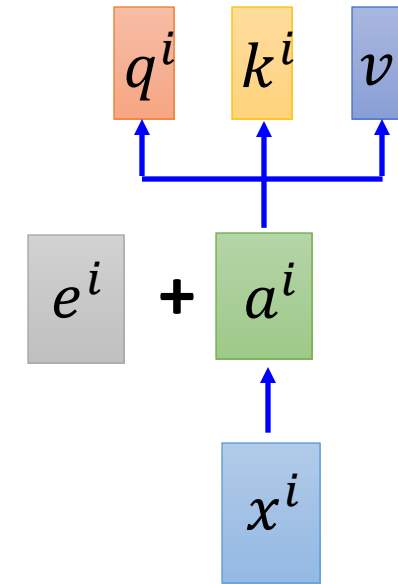
(2 heads as example)

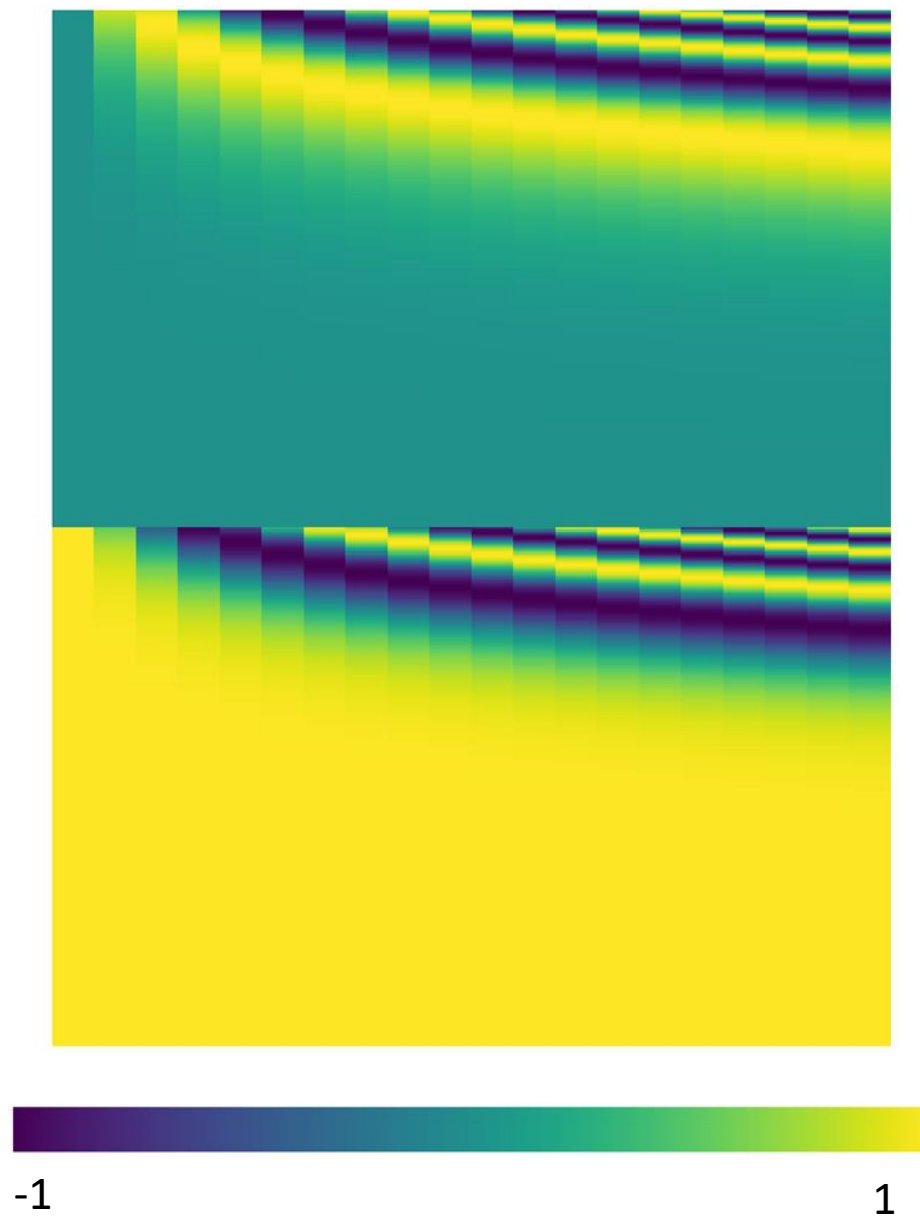
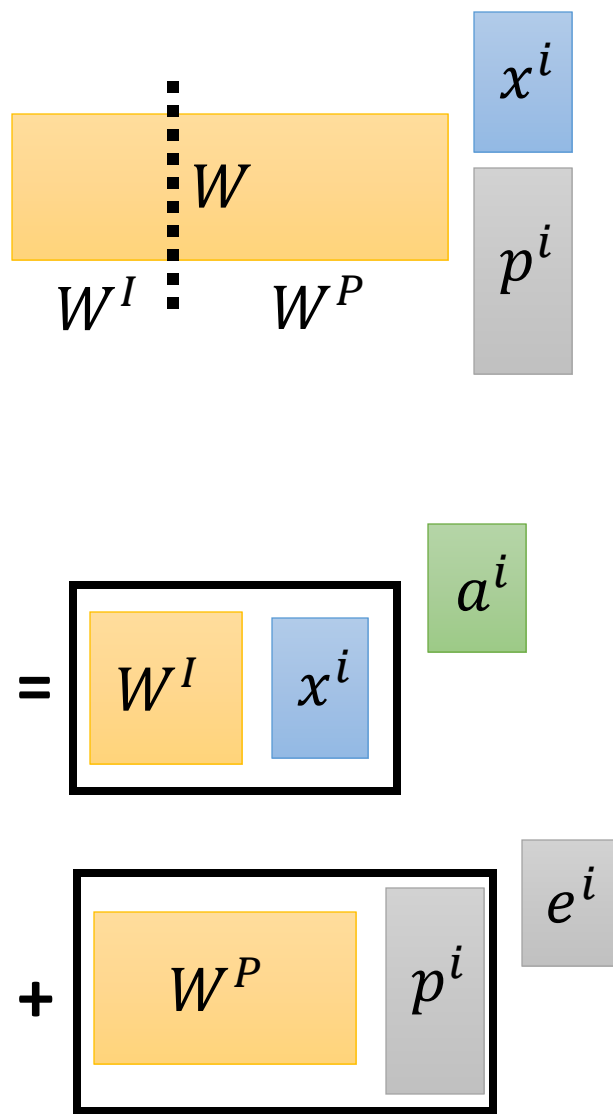
$$b^i = W^O \begin{bmatrix} b^{i,1} \\ b^{i,2} \end{bmatrix}$$



Positional encoding

- No position information in self-attention.
- Each position has a unique positional vector e^i (not learned from data)
- Each x^i appends a one-hot vector p^i



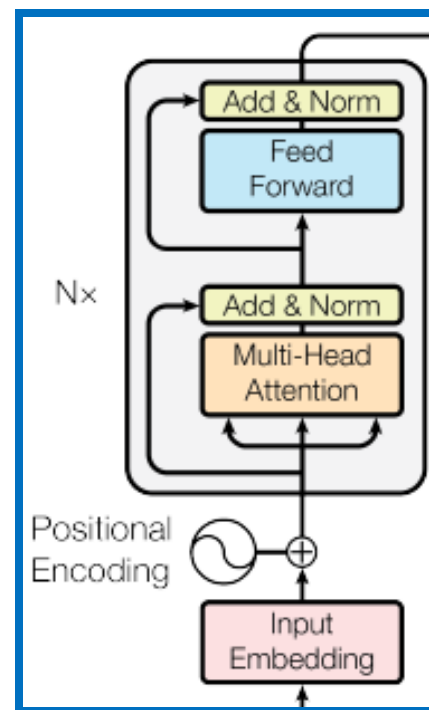


Transformer

Using Chinese to English translation as example

Both encoder and decoder consist of self-attention layers

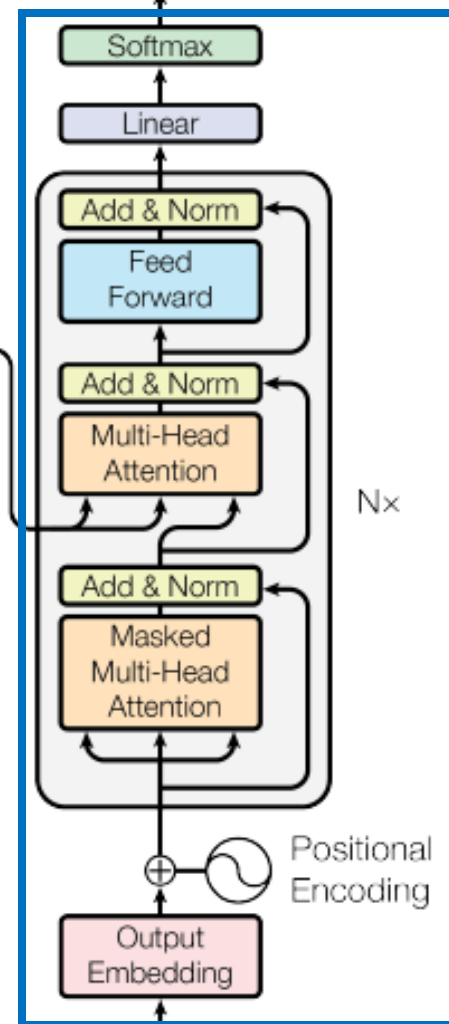
Encoder



Inputs

机器学习

machine learning



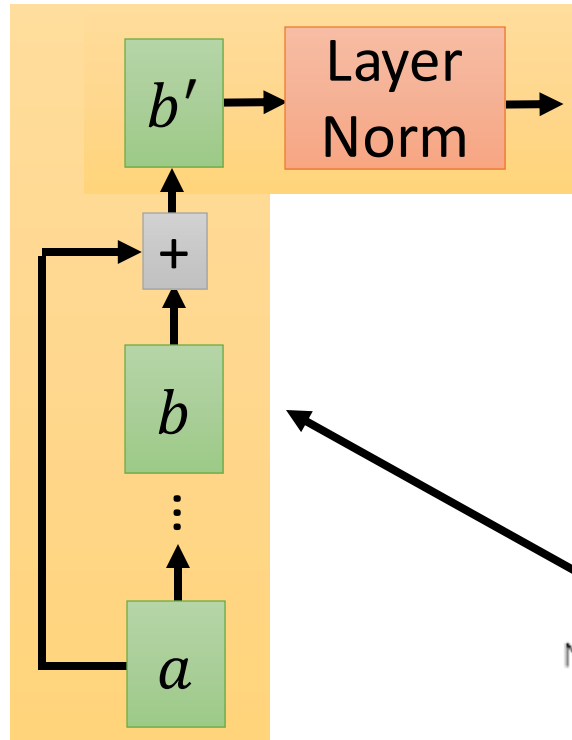
Outputs
(shifted right)

<BOS>

machine

Decoder

Transformer

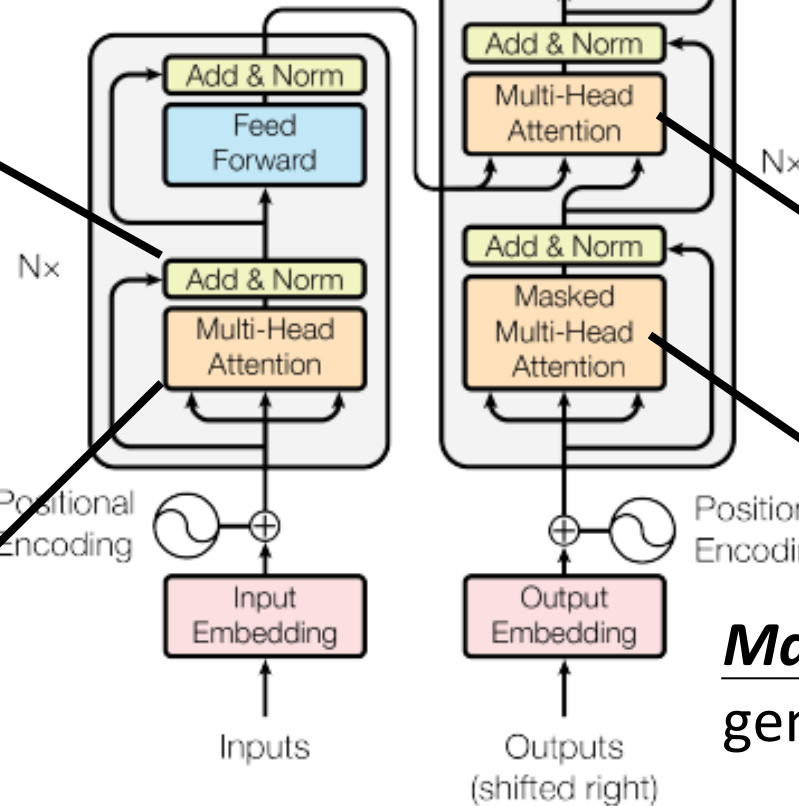
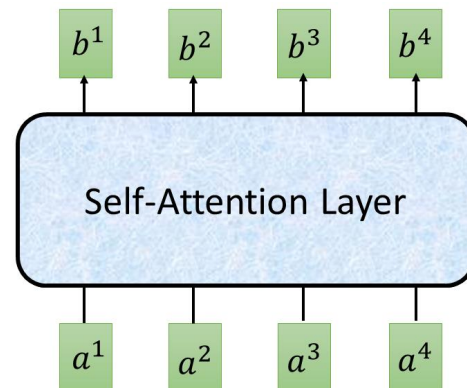


Layer Norm:

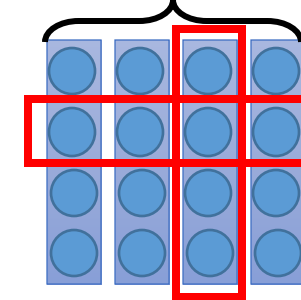
<https://arxiv.org/abs/1607.06450>

Batch Norm:

<https://www.youtube.com/watch?v=BZh1ltr5Rkg>



Batch Size



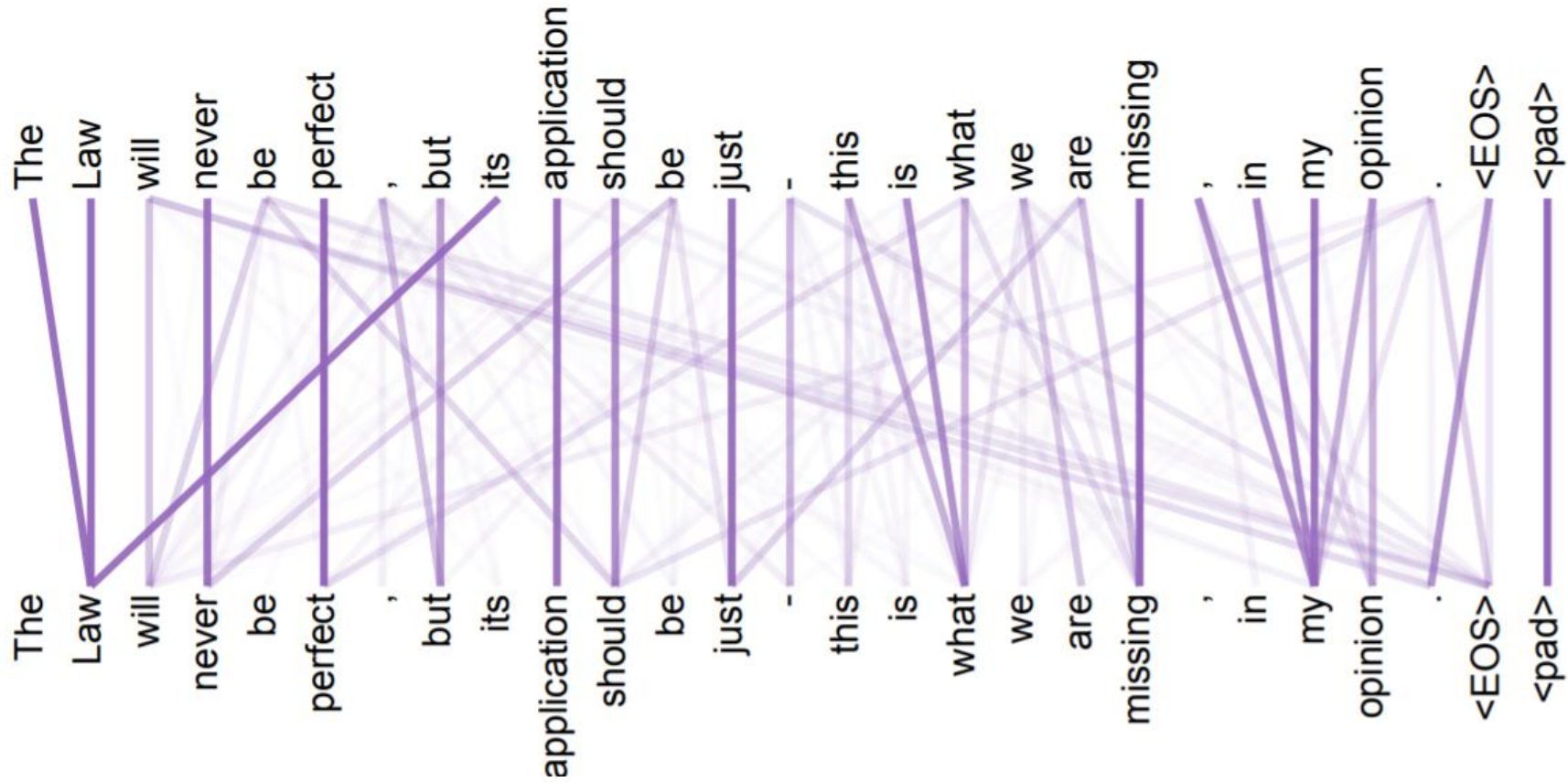
$\mu = 0,$
 $\sigma = 1$
Batch

$\mu = 0, \sigma = 1$
Layer

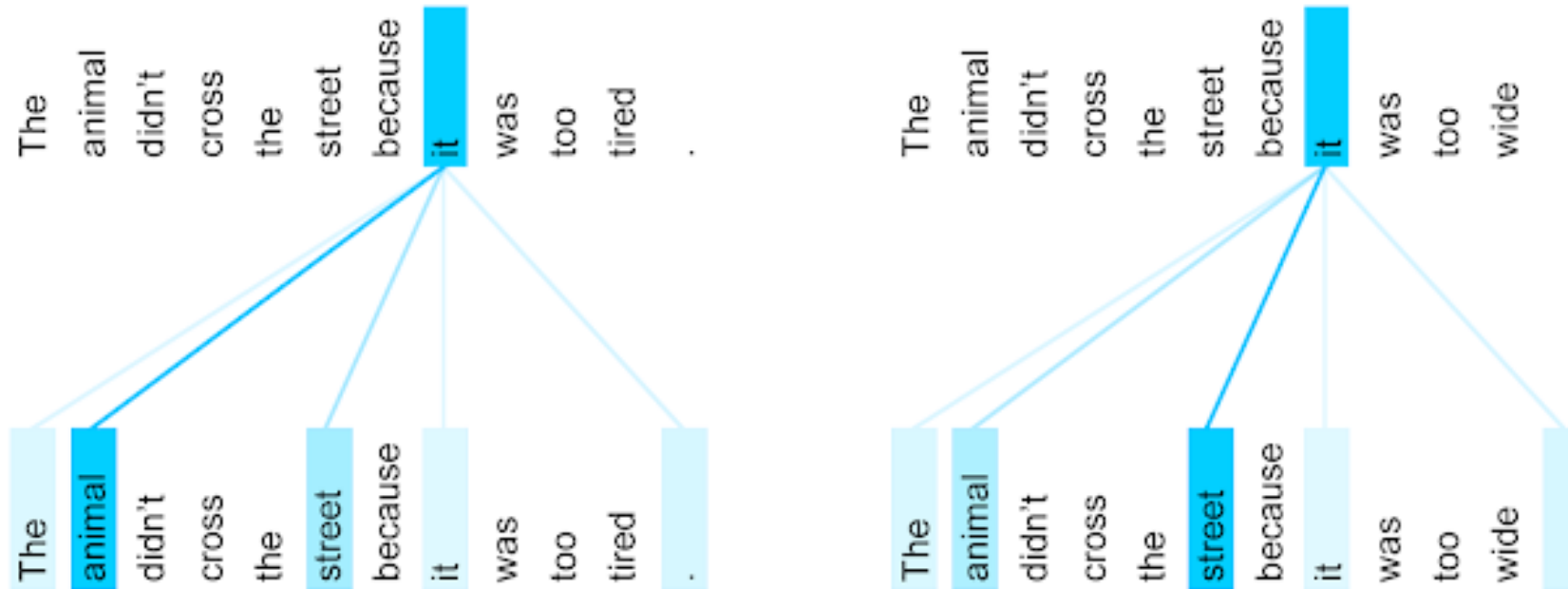
attend on the
input sequence

Masked: attend on the
generated sequence

Attention visualization



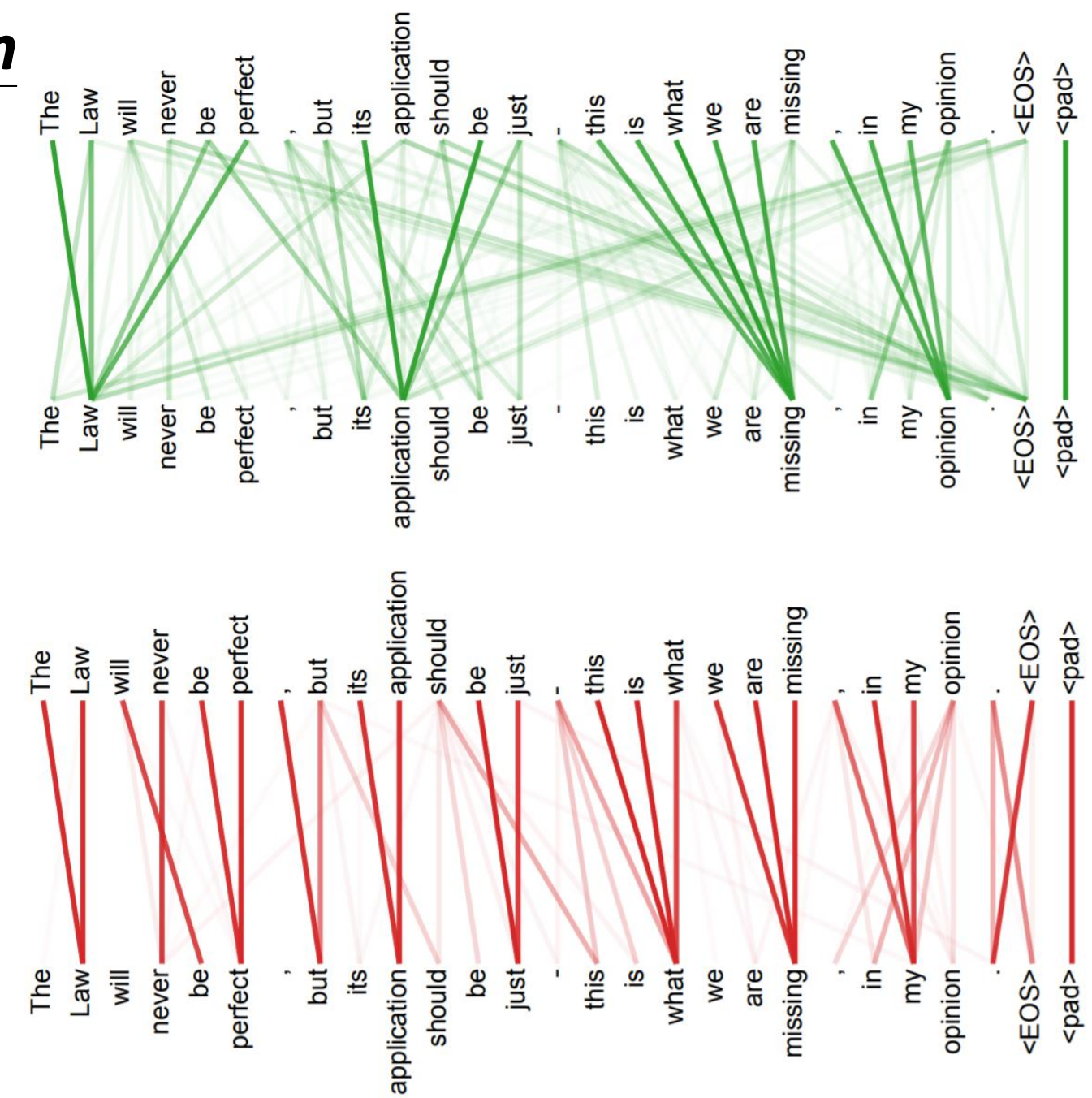
Attention visualization



The encoder self-attention distribution for the word "it" from the 5th to the 6th layer of a Transformer trained on English to French translation (one of eight attention heads).

<https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html>

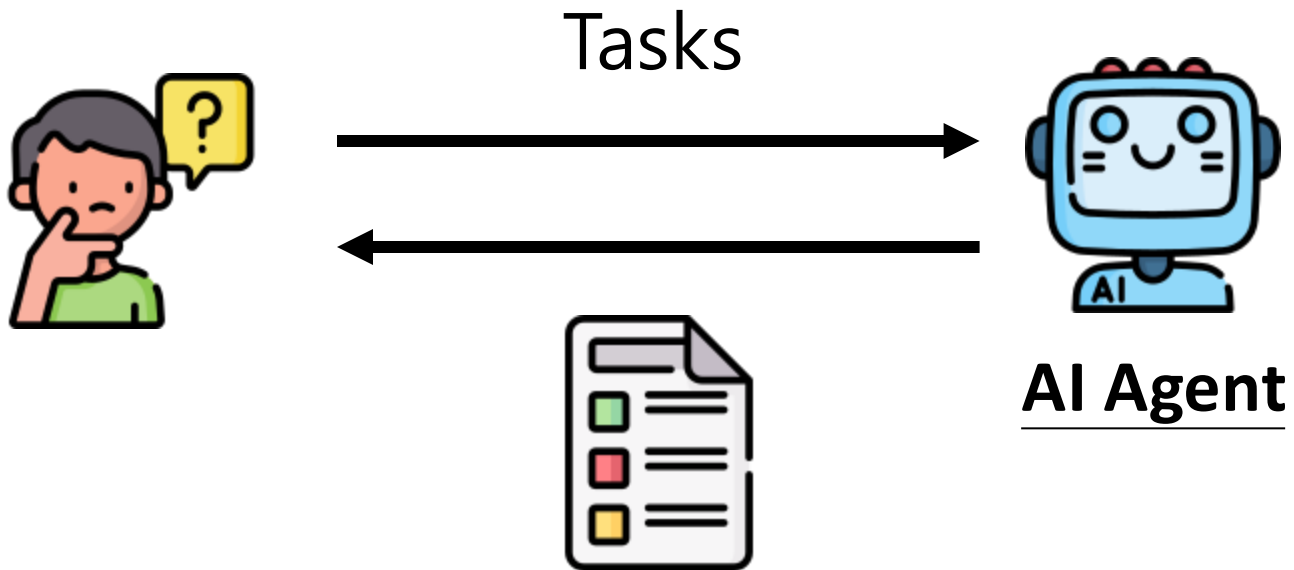
Multi-head Attention



Agenda

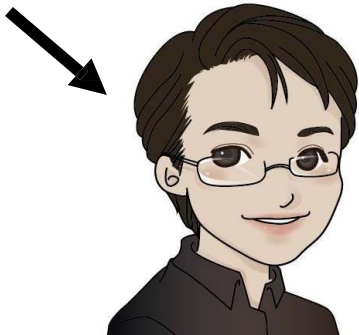
- Self-Attention Networks (Transformer)
 - Why use self-attention for seq2seq processing
 - Transformer network details
- Building Agent with Large Language Models

How most people use AI today



Humans can perform complex tasks that require multiple steps

Goal: Host a dinner party with friends



Plan : (1) Survey availability, (2) Collect results, (3) Book Restaurant A

Use Google Forms to survey availability

Fill out the form

Decide on Thursday evening

Try to book Restaurant A

No availability on Thursday evening

Search Google Maps

Restaurant recommendations

Switch to booking Restaurant B

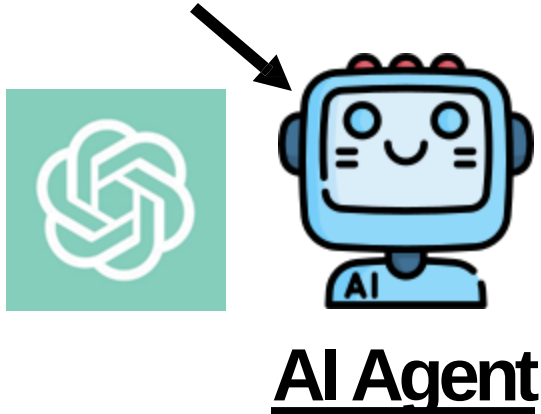


Takeaway:

- Multi-step tasks require planning
- Plans often can't keep up with changes

Humans can perform complex tasks that require multiple steps

Goal: Host a dinner party with friends



Plan : (1) Survey availability, (2) Collect results, (3) Book Restaurant A

Use Google Forms to survey availability

Fill out the form

Decide on Thursday evening

Try to book Restaurant A

No availability on Thursday evening

Search Google Maps

Restaurant recommendations

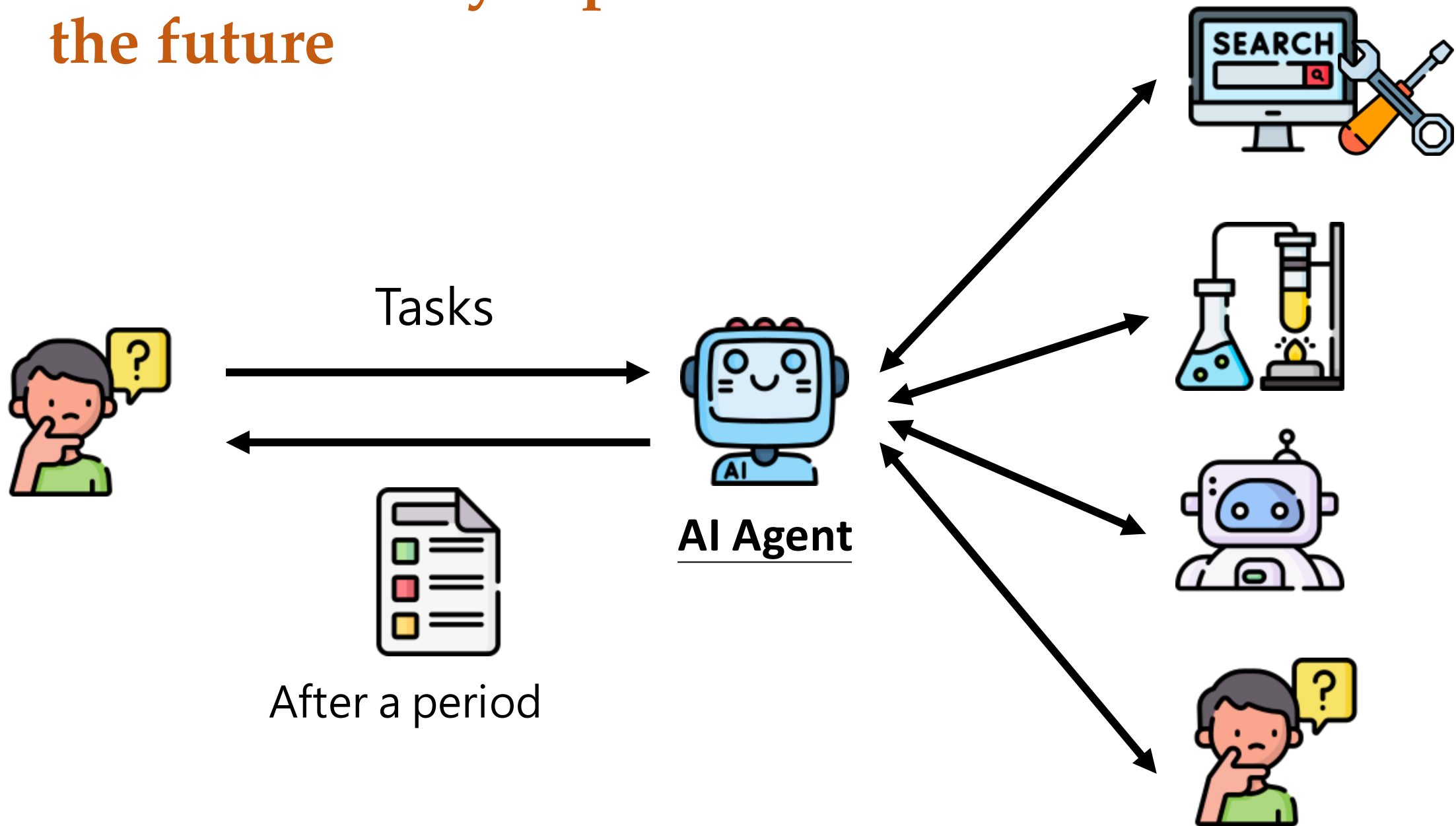
Switch to booking Restaurant B



Takeaway:

- Multi-step tasks require planning
- Plans often can't keep up with changes

What humanity expects from AI in the future



AI Agent

- AutoGPT: <https://github.com/Significant-Gravitas/Auto-GPT>
- AgentGPT: <https://agentgpt.reworkd.ai/>
- BabyAGI: <https://github.com/yoheinakajima/babyagi>
- Godmode: <https://godmode.space/?ref=futuretools.io>

A virtual village composed of AI villagers

<https://youtu.be/44TH6uKINC4?si=6F3gTukuvqSE3pEb>

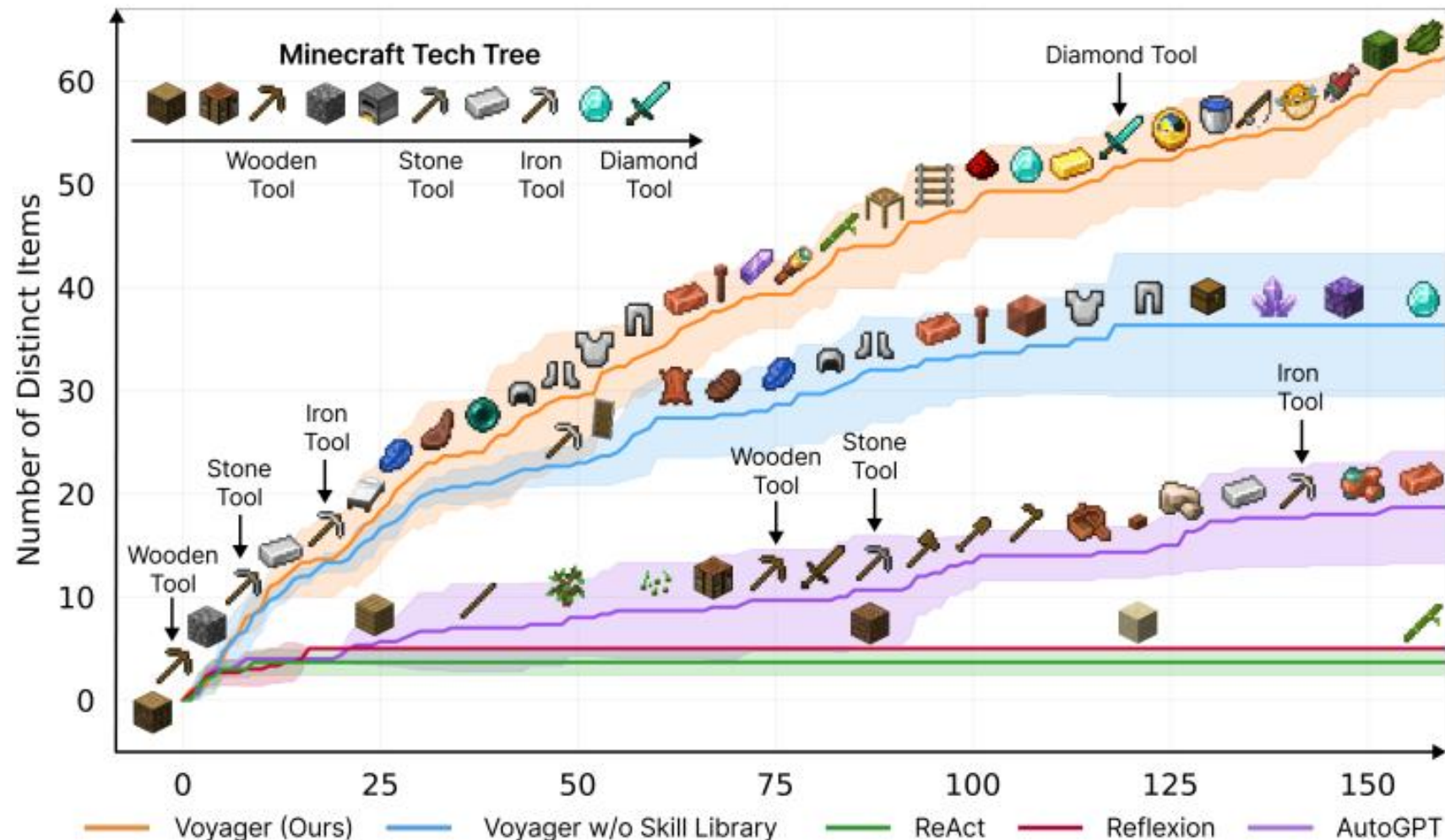
<https://youtu.be/XY5Wncq5vAE?si=8-b1gTBbfN4c3c2Q>

<https://arxiv.org/abs/2304.03442>



An AI that can play Minecraft on its own

Voyager: <https://arxiv.org/abs/2305.16291>



Robots controlled by language models

Figure 01



Figure Status Update - OpenAI Speech-to-Speech Reasoning

<https://youtu.be/Sq1QZB5baNw?si=-TEsPeUqTvhIS60E>

Robots controlled by language models

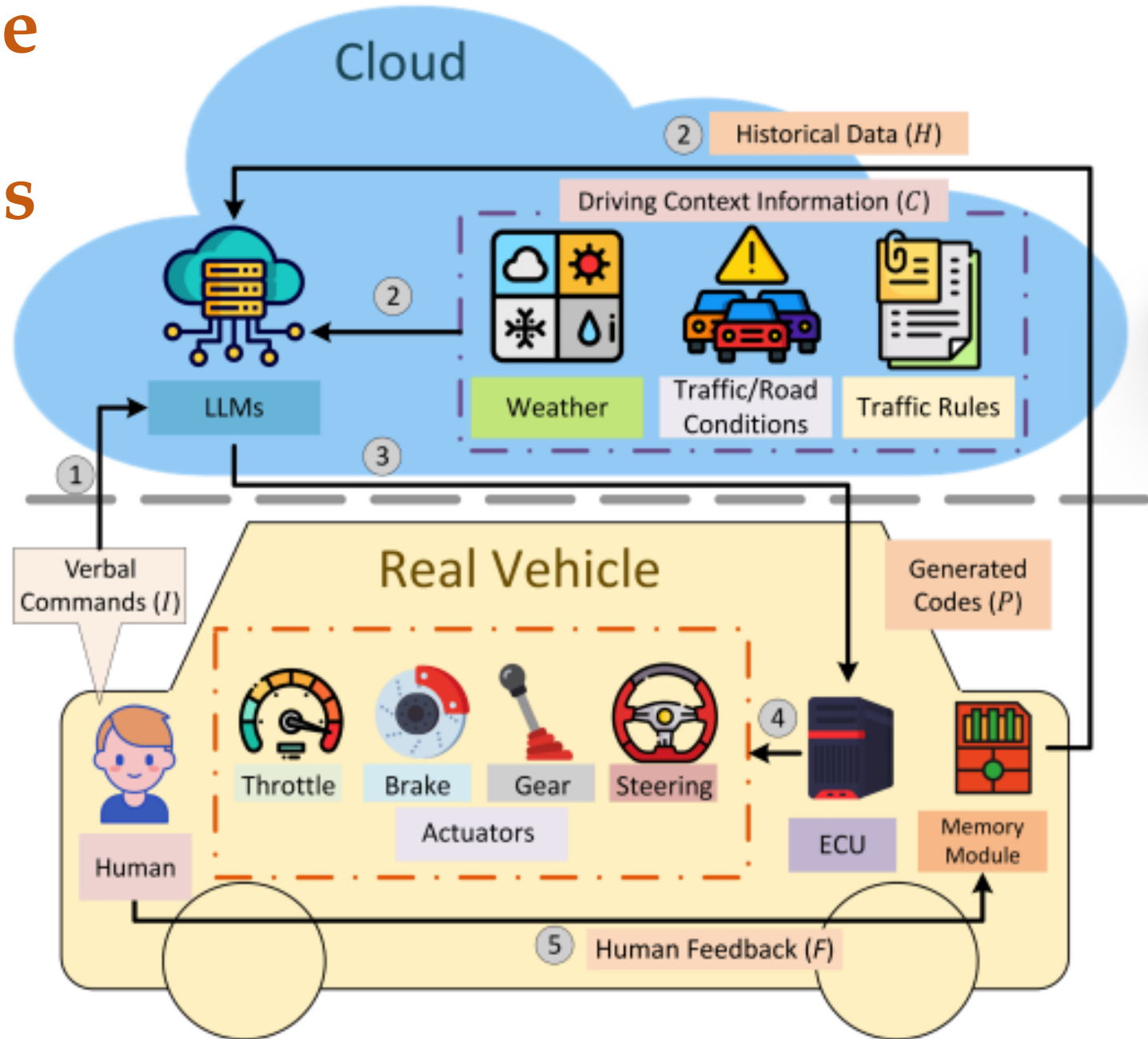
<https://innermonologue.github.io/>

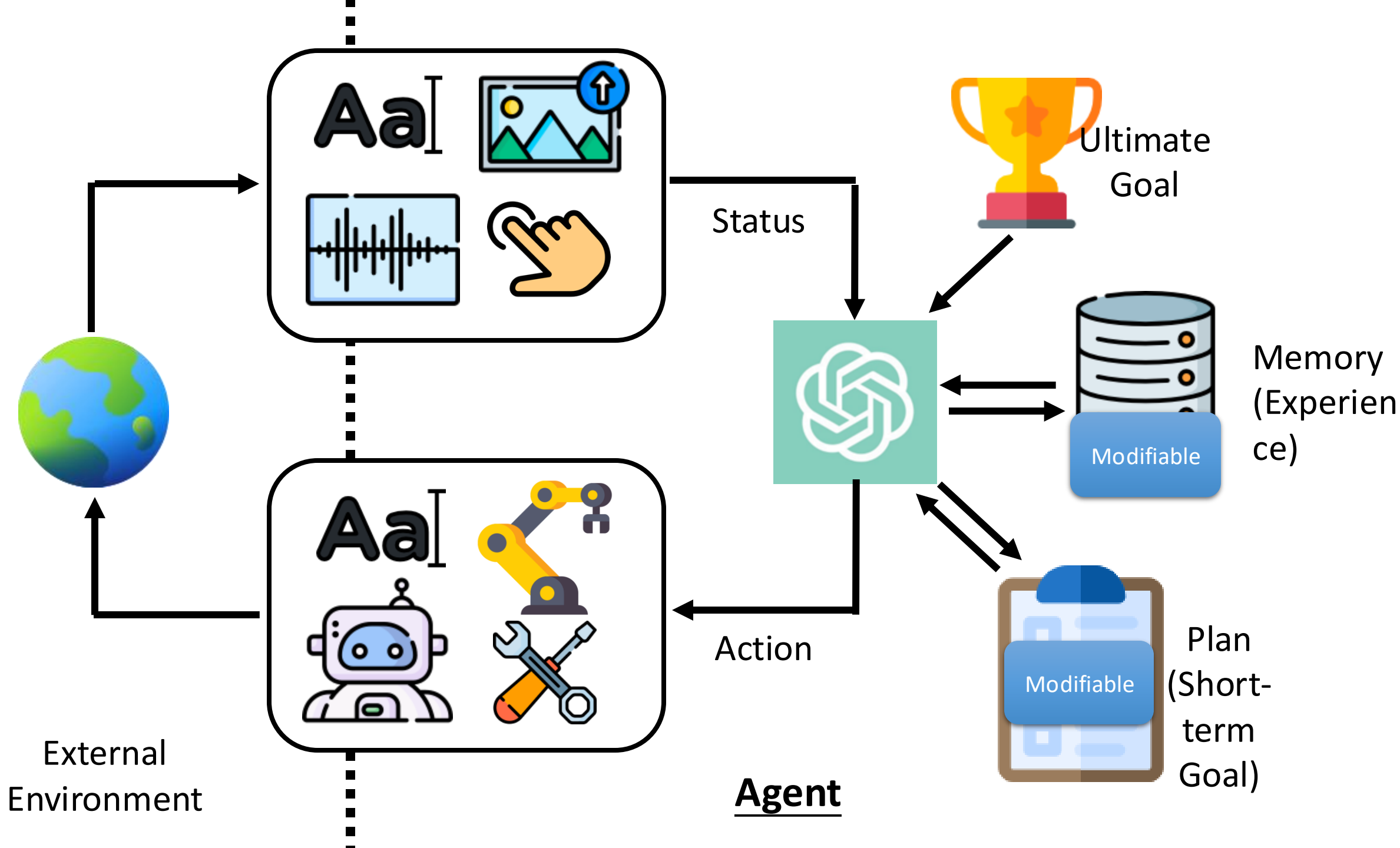
<https://arxiv.org/abs/2207.05608>

Using large language models to drive autonomous vehicles

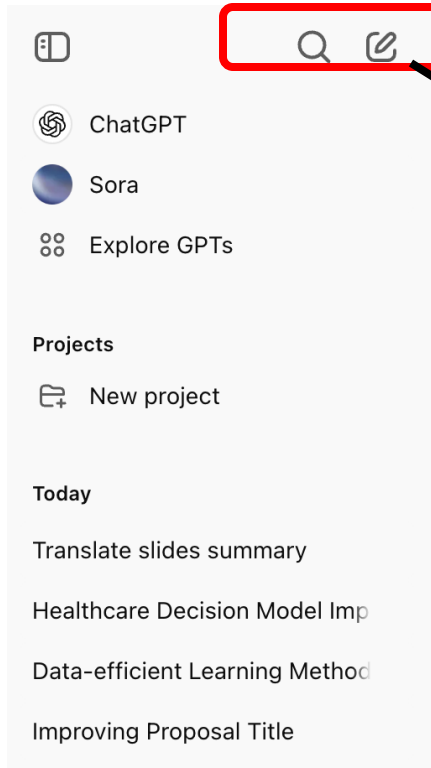
Talk2Drive

<https://arxiv.org/abs/2312.09397>





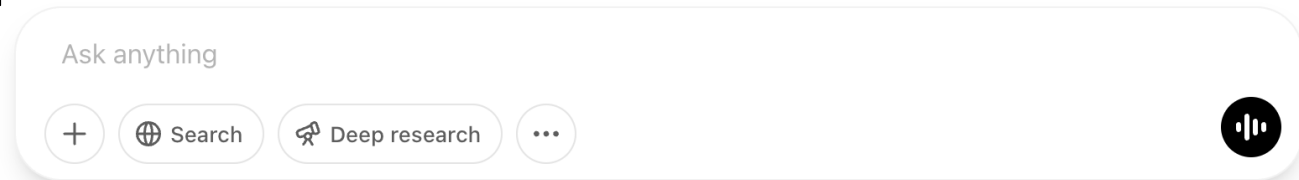
ChatGPT with memory



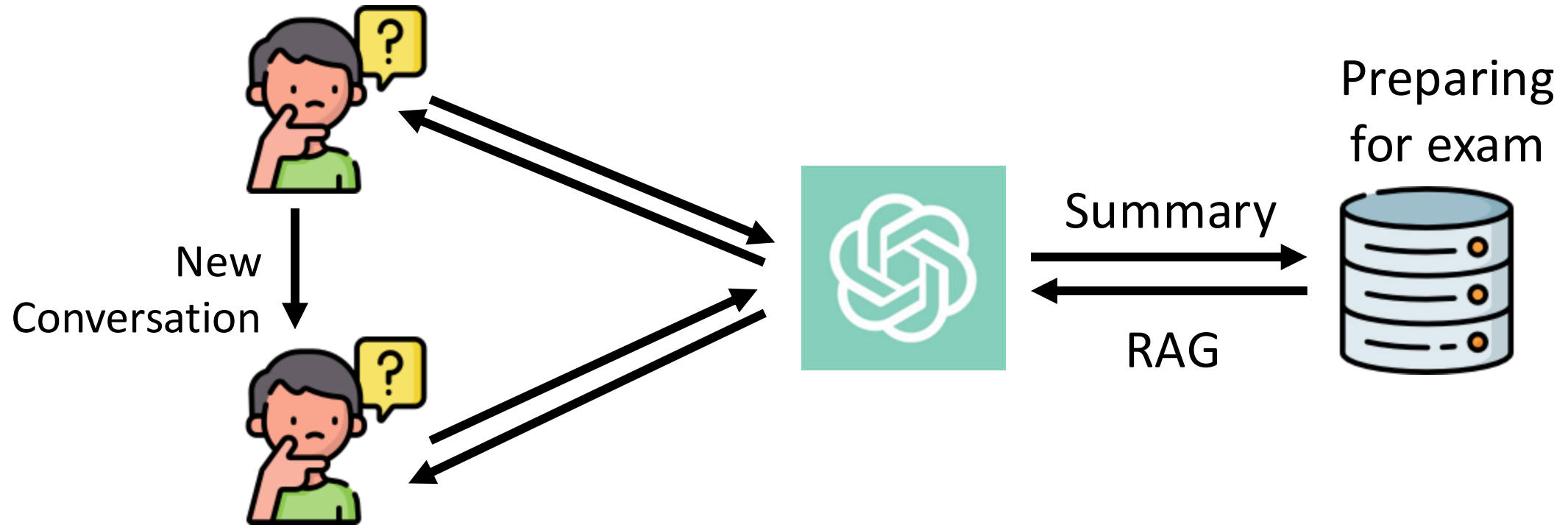
ChatGPT 4o ▾

**Every time a new
conversation
starts, everything
starts from
scratch**

What can I help with?



ChatGPT with memory



Open AI is working on it?

<https://openai.com/blog/memory-and-new-controls-for-chatgpt>



<https://github.com/cpacker/MemGPT>

Further Reading

Overview paper: <https://arxiv.org/abs/2309.07864>

